

1 Topic

1.1 Background and Need

Sudden collapses of aging bridges — a recurring social problem

In April 2023, the walkway of the roughly 30-year-old Jeongja Bridge in Bundang collapsed during ordinary pedestrian use after corrosion and damage accumulated in its cantilever members, killing one person and injuring another. In January 2022, the Fern Hollow Bridge in Pittsburgh, USA, also collapsed during the morning commute after corrosion had been flagged in inspections but repairs were delayed. In both accidents, warning signs observed in inspections did not lead to repair or reinforcement, showing that periodic inspection alone cannot track changes in structural condition continuously enough to prevent accidents.



[Fig. 1: Jeongja Bridge collapse site]



[Fig. 2: Fern Hollow Bridge collapse]

As in these two cases, the corrosion, cracking and fatigue that accumulate over years in aging bridges change dynamic properties such as stiffness, damping and natural frequency, so they can be observed through vibration-based monitoring. Vibration characteristics, however, change not only with damage but also with environmental variables such as temperature and humidity, making it hard to tell whether an observed change comes from damage or from the environment. Two tasks remain: capturing changes in dynamic properties continuously between inspections, and separating the environmental effect from those changes so that only damage-related anomalies are flagged.

According to the Korea Authority of Land & Infrastructure Safety's 2025 statistics by facility class (classified by size), Korea has 37,543 bridges — 5,392 Class 1, 7,925 Class 2 and 24,226 Class 3 — and the relatively small Class 3 bridges account for about 64.5%. Because so many structures must be managed, the need for low-cost, continuous screening that also works on small and medium-sized bridges keeps growing.



[Fig. 3: Share of Class 3 bridges among all bridges in Korea]

The continuous-monitoring blind spot of small and medium bridges

Independently of its size, a bridge's condition is rated by safety grade (A-E), and from December 2025 precision safety diagnosis was extended to aging and low-grade (C·D·E) facilities as well. Precision safety diagnosis, however, is costly and labor-intensive and is performed only on a fixed cycle — every 4 to 6 years depending on the safety grade — so a time gap remains between inspections, and changes in dynamic properties that accumulate during that gap are

hard to detect. As a result, aging small and medium bridges such as pedestrian overpasses and small footbridges lack a low-cost supplementary means of continuously detecting changes in condition between inspections.

종류\안전등급	A등급	B·C등급	D·E등급
정기안전점검	반기 1회 이상	반기 1회 이상	1년 3회 이상
정밀안전진단	6년 1회 이상	5년 1회 이상	4년 1회 이상

[Fig. 4: Inspection cycles and targets by facility grade]

No low-cost screening technology before precision diagnosis

Existing SHM (Structural Health Monitoring) systems install sensors on a structure permanently for continuous measurement, but they rely on precision equipment and dedicated infrastructure — ultra-precise piezoelectric accelerometers, high-performance data-acquisition hardware, distributed fiber-optic sensors (e.g., KRISS BOCDA) — and are therefore applied mainly to a few large, critical bridges such as Incheon Bridge; their cost and installation constraints make them impractical to deploy widely on small and medium aging bridges in service. Between periodic precision diagnosis and costly continuous measurement, low-cost continuous screening that precedes precision diagnosis is effectively missing. Bridge safety management therefore needs a multi-stage concept in which continuous screening (first stage) filters out anomaly candidates before precision safety inspection and diagnosis, so that limited inspection staff and budgets can be concentrated on suspect bridges.

1.2 Topic and Purpose

This work addresses the three problems summarized above: ① capturing changes in dynamic properties continuously between inspections while separating environmental effects, ② the time gap between precision-diagnosis cycles, and ③ the difficulty of applying precision measurement systems to small and medium aging bridges. It is not a system that predicts collapse or replaces precision safety diagnosis; it is a low-cost first-stage (supplementary) screening system that fills the time gap between inspections and diagnoses by indicating which bridges should be inspected first.

The purpose of this work is to develop a low-cost, machine-learning-based bridge vibration screening system that addresses this blind spot and this missing technology at the same time, with three goals. First, vibration data are collected with an ADXL355Z MEMS accelerometer and a Raspberry Pi (3 Model A+ in stage 1, 5 in stage 2); because damage data cannot be created deliberately on a bridge in service, unsupervised learning trained only on normal data is used. Second, since a bridge's natural frequency rises and falls with air temperature and humidity even without damage [2], a DHT22 records temperature and relative humidity, and the normal temperature/humidity-natural-frequency relationship is learned so that variation explained by the environment is removed and false alarms are reduced. Third, because an anomaly signal alone does not tell an inspector where to look, the system implements explainable alarms that also show which frequency bands deviated from the normal baseline, guiding the direction of the follow-up precision inspection.

Approach

The three goals are implemented as a single unsupervised anomaly-detection pipeline. 1) Only the vibration patterns of the normal state are learned; 2) after normal variation explained by temperature and humidity is removed first, a residual that deviates strongly from the learned pattern is judged an anomaly candidate; and 3) the system visualizes which frequency bands have a large reconstruction error (the difference between the input and its reconstruction from the learned normal pattern, which grows as the input departs from normal — see 2.3), providing interpretive information that supports the choice of inspection direction rather than a definitive diagnosis. The hardware is described in 2.2, and the concrete procedure that links the three functions is covered step by step in 2.3 Operating Principle.

1.3 Similar Products (Research) and Differentiation

SHM systems based on low-cost MEMS accelerometers

An SHM study combined a Raspberry Pi with a low-cost MEMS accelerometer (ADXL355) and showed that a structure's natural frequencies, mode shapes and damping can be identified without ultra-precise piezoelectric sensors costing tens of millions of won [1]. That study, however, focuses on identifying dynamic characteristics and does not address how environmental variables such as temperature and humidity, or changes in operational loading, affect anomaly detection.

Distributed fiber-optic sensor systems

The BOCDA-based fiber-optic sensor system of the Korea Research Institute of Standards and Science (KRISS) precisely measures strain and load position in a structure with high spatial resolution [4]. As precision strain measurement that presupposes dedicated infrastructure, it differs from this work's vibration-based continuous screening in both the physical quantity measured and its purpose.

Unsupervised-learning SHM algorithms

The Swiss Z24 Bridge Benchmark (provided by KU Leuven) is a representative public dataset that includes long-term monitoring and progressive damage-test data [3]. Many unsupervised models — Autoencoder, One-Class SVM, Isolation Forest, VAE and others — have been compared on Z24 [5][6], but most stop at outputting an anomaly score or a threshold exceedance and do not explain which band departed from the baseline.

Environmental-compensation research on long-term data from a full-scale bridge

The openLAB Research Bridge public dataset, from the long-term monitoring of a full-scale research bridge in Bautzen, Germany, provides acceleration responses together with environmental measurements such as air temperature and humidity, making it well suited to analyzing seasonal and daily temperature effects (details in 2.7) [8]. In this work, openLAB serves as supplementary validation data for checking whether the environmental-compensation principle also holds on full-scale long-term data; the core contribution lies in a low-cost hardware implementation that links collection, compensation, judgment and explanation in one pipeline, and in validation with our own model bridges, which distinguishes it from studies that reanalyze public data.

The distinctiveness of this work therefore rests on three axes: direct low-cost hardware implementation, temperature/humidity compensation, and band-wise explainability. Unlike low-cost MEMS studies, it reduces false alarms through temperature/humidity compensation; unlike fiber-optic measurement, it performs continuous screening with inexpensive general-purpose parts; and unlike unsupervised studies that output only an anomaly score, it also reports the contribution of each band to the reconstruction error.

2 Building the System

2.1 Objective

This system is a supplementary vibration-screening system for early detection of vibration-anomaly candidates in aging small and medium bridges, based on low-cost, low-power hardware and unsupervised learning. It collects vibration, temperature and relative humidity with an ADXL355Z-based MEMS accelerometer and a DHT22 temperature/humidity sensor, and aims to reduce environment-driven false alarms using the temperature/humidity-frequency relationship learned in the normal state. It then detects anomaly candidates that depart from the normal vibration pattern with Autoencoder-based unsupervised learning and, when an anomaly occurs, shows which frequency bands (or bands around modes) have large reconstruction errors, helping to prioritize precision inspection.

Our own experimental validation has two stages. Stage 1 confirms that the whole collection-compensation-detection-explanation pipeline works, using a foam-board model and manual excitation; stage 2 overcomes stage 1's two limitations (indistinct resonance peaks and variation in manual excitation) with an aluminum multi-span model and

a solenoid auto-tapper, validates discrimination performance on a matrix of temperature, humidity, mass and bolt-loosening conditions, and builds a real-time measurement and analysis setup based on a Raspberry Pi 5 (details in 2.2 and 3.1).

2.2 Design

Hardware

The system is built as a hardware acquisition unit and a software analysis unit. The hardware components and the reasons for each design choice are listed in the table below.

Component	Role · design rationale
ADXL355Z 3-axis accelerometer	3-axis vibration — detects changes in natural frequency and other vibration traits
DHT22 temperature/humidity sensor [7]	Temperature/humidity — compensates for environment-driven normal variation
Raspberry Pi 3 Model A+ (stage 1)	Data collection and storage (Python logging)
Raspberry Pi 5 (stage 2)	Runs real-time acquisition, tapper control, FFT, anomaly score and dashboard on one device
Foam-board bridge model (stage 1)	First model for structural-condition tests such as mass distribution
Aluminum bridge model (stage 2)	Low damping gives sharp resonance peaks; bolted joints give repeatable boundary conditions
Solenoid auto-tapper (stage 2)	Automatic excitation at fixed force, period and position — removes manual variation and narrows normal scatter

The core measurement and processing hardware — ADXL355Z evaluation board ₩84,589, DHT22 sensor ₩5,680 and Raspberry Pi 5 (about ₩100,000 for the 2-4 GB RAM model) — costs about ₩190,000 in total. This is far below existing SHM measurement that presupposes ultra-precise piezoelectric accelerometers and dedicated acquisition equipment (based on piezoelectric sensors costing tens of millions of won; see 1.3 and [1]), supporting this work's goal of a low-cost configuration that can be deployed on many small and medium bridges.

Software

The software analysis unit consists of two programs, one for acquisition and one for analysis. The acquisition program runs in Python on the Raspberry Pi and writes the ADXL355Z 3-axis acceleration at 200 Hz (500 Hz in stage 2) and the DHT22 temperature and relative humidity at about 2-second intervals to a single CSV file. The analysis program splits this file into time windows and performs, in order, FFT and feature extraction, residual calculation by multivariate regression on temperature T and relative humidity RH , standardization, reconstruction-error-based anomaly scoring, a 99th-percentile threshold decision, and band-contribution output.

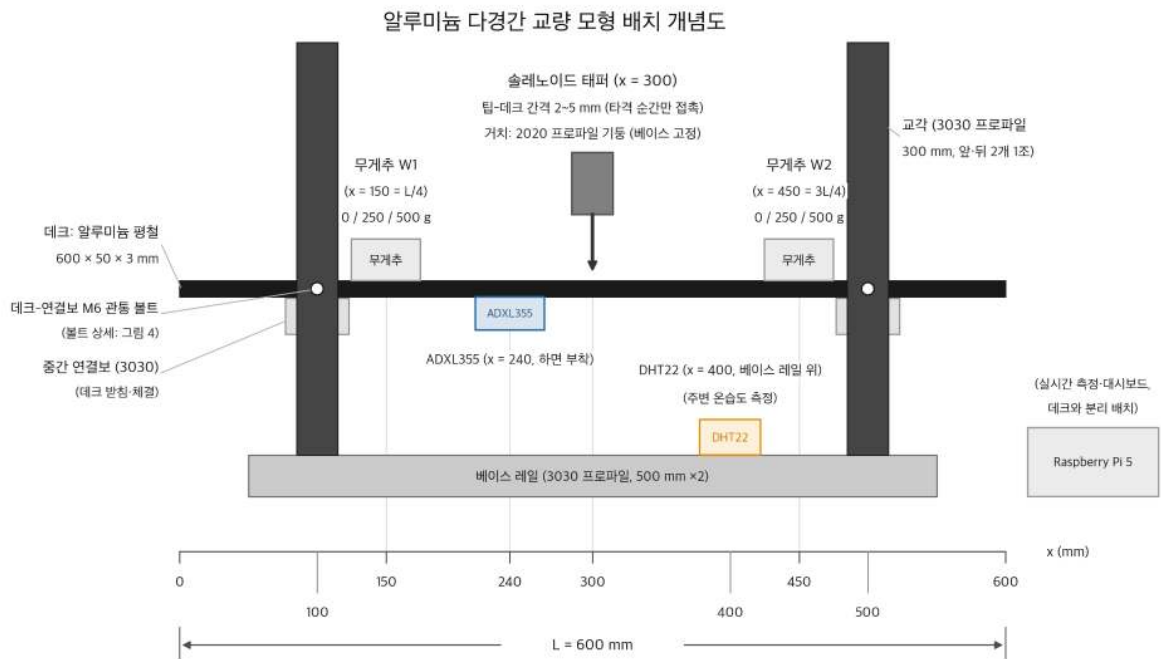
In stage 2 this configuration is extended into a real-time system on a Raspberry Pi 5. The acquisition logger, tapper control and analysis server run on the same device to guarantee a shared clock; the window right after each tap (from $t_0 + 0.05$ s for 2.0 s, where t_0 is the tap time) is cut out, and its spectrum and anomaly score are computed immediately and shown on a web dashboard. The dashboard refreshes about every 0.7 s with the live z-axis waveform, tap spectrum, ratio to threshold (anomaly score $\div$ threshold), peak frequency, RMS, temperature/humidity and band contributions; a ratio below 0.8 is shown as normal, 0.8-1.0 as caution, and 1.0 or above as an anomaly candidate. The real-time decision is for on-site monitoring; all raw data, including condition codes, are saved to CSV and used for model training and evaluation.

Sensor placement

On the stage-1 foam-board model, the ADXL355Z is mounted at mid-span, where the first bending response is largest ($x = L/2$, $y = B/2$ for span L and width B), and the vertical z axis, which reflects vertical bending, is the main axis of analysis.

On the stage-2 aluminum model, every position is defined along the length of the deck (overall length 600 mm) with $x = 0$ at the left end. The solenoid tapper strikes the centerline of the top surface at mid-span, $x = 300$ mm, so that every tap consistently excites the first mode, and the gap between tip and deck is held at 2-5 mm with a card gauge in all conditions so that they touch only at the moment of impact. The ADXL355Z is attached to the underside of the deck at $x = 240$ mm, away from directly below the impact point, to measure the large bending response near mid-span while avoiding the local impact component right after the tap and interference from the weights and the impact point on top. Weights are placed at $x = 150$ mm ($L/4$) and $x = 450$ mm ($3L/4$), the same proportions as in stage 1.

In both experiments the DHT22 and the Raspberry Pi are kept off the vibrating structure (deck and piers) so that they do not affect the bridge's mass or stiffness. In stage 1 they were placed beside the model with the cables fixed loosely; in stage 2 the DHT22 is fixed on the base rail at $x = 400$ mm so that temperature and relative humidity around the bridge are recorded at the same position in every condition, and the Raspberry Pi sits beside the base, away from the deck, so that cable tension and mass do not influence the vibration response.



[Figure 1: Layout of the aluminum multi-span bridge model (stage 2)]

2.3 Operating Principle

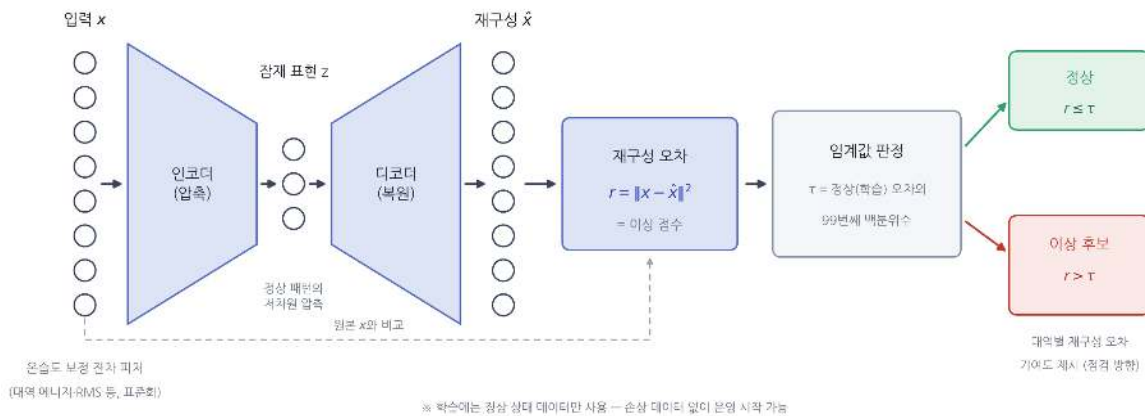
1. Vibration and temperature/humidity acquisition: 3-axis acceleration is collected from the ADXL355Z, and temperature T and relative humidity RH from the DHT22. The actual natural-frequency range was checked in a preliminary test so that the analysis band stays within the Nyquist limit, and the plan was to raise the sampling rate to 500 Hz if the measured first natural frequency approached or exceeded the 100 Hz Nyquist limit of 200 Hz sampling. On the stage-2 aluminum model, preliminary measurement after assembly showed the first natural frequency above that limit, so the sampling rate was fixed at 500 Hz and the measurement range at ± 8 g (measured natural frequencies are given in 3.4). Excitation is by manual tapping in stage 1 and by automatic taps from the solenoid auto-tapper in stage 2 (at a fixed period after warm-up); tap times are logged on the same clock, and only the free-vibration segment right after each tap is used as the unit of analysis.

2. Signal preprocessing and frequency-band feature extraction: the DC offset is removed from the acceleration signal and the spectrum is estimated (FFT/STFT in stage 1; in stage 2 the Welch method, which reduces noise by averaging spectra over short segments). RMS, standard deviation, peak-to-peak, peak frequency and power, band energies and so on are computed per segment and axis, and the analysis bands are reset to match the measured first natural frequency (stage 1: 0-12, 12-25, 25-50 Hz based on the preliminary 22.6 Hz peak; stage 2: 0-40, 40-80, 80-125 Hz based on the measured 101 Hz — see 3.4). The final stage-2 implementation uses eight z-axis features (log RMS, log p2p, peak

frequency, log peak power, log total energy and three band-energy ratios), with amplitude/energy features log-scaled and band energies normalized as shares of the total (rationale in 3.5). Temperature T and relative humidity RH are used for environmental compensation. The z-axis peak frequency serves as the indicator of the first natural frequency, and the band-wise analysis is an auxiliary indicator that explains which vibration characteristic departed from the baseline, not a way to locate damage.

3. Multivariate regression compensation of the normal natural frequency by temperature and humidity: on normal data, a multivariate regression that predicts the first natural frequency from temperature T and relative humidity RH is learned; at inference, the difference between the measurement f and the prediction $\hat{f} = g(T, RH)$, $r = f - \hat{f}$, is defined as the residual corrected for the temperature/humidity effect. The same regression and residual calculation are applied to every vibration feature, and the resulting residual features are the input to the anomaly-detection model. Because the residual can still contain sensor noise, mounting condition and operational loading besides structural anomaly candidates, it is used as a criterion for detecting anomaly candidates, not as a confirmation of damage.

4. Unsupervised anomaly detection with an Autoencoder: the Autoencoder is trained only on standardized residual features from the normal state (architecture in Figure 2). The model compresses the input into a low-dimensional latent representation and reconstructs it, learning only the correlation structure of normal data, so data that depart from the normal pattern have a larger reconstruction error. The threshold is set at the 99th percentile of the reconstruction error on normal validation data (in stage 2, the error distribution of 90 normal windows held out exclusively for the threshold; see 3.2). Because both training and thresholding use only normal data, operation can begin without any damage data. The stage-1 implementation used the PCA reconstruction error, equivalent to a linear Autoencoder; stage 2 adopted a nonlinear Autoencoder with an 8-3-8 hidden structure (tanh, early stopping).



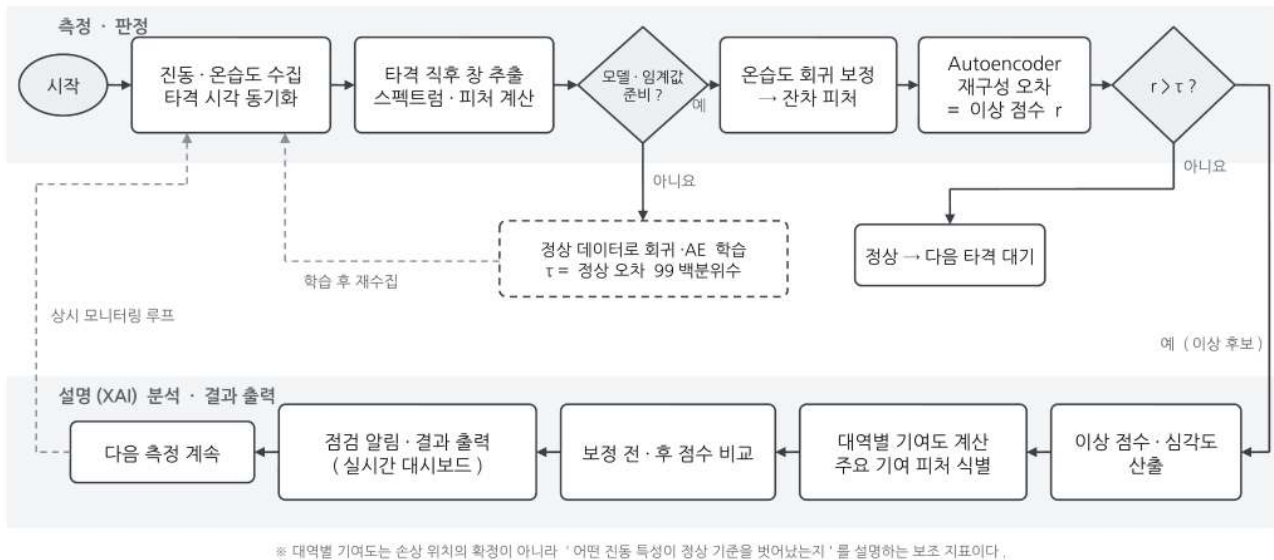
[Figure 2: Autoencoder-based anomaly detection]

5. Band-wise reconstruction-error contribution: when an anomaly is detected, the system also analyzes which frequency bands (or bands around modes) have large reconstruction errors. A contribution is computed as “that band's reconstruction error ÷ total reconstruction error”; SHAP is kept as an optional feature that can additionally be applied to an auxiliary model, and band contribution is the default method of explanation.

6. Output and precision-inspection alerts: when the anomaly score exceeds the threshold, the system raises an alarm and outputs the anomaly score, a before/after comparison of temperature/humidity compensation, the main contributing bands, the rate of change in the first natural frequency, the band contributions and an interpretation for inspectors. Separately from whether the threshold is exceeded (detection), the ratio to threshold (2.2) and the share of recent windows exceeding the threshold are computed as severity indicators and presented as grounds for prioritizing precision inspection; because these indicators express the degree of departure from the normal state, not the physical severity of damage, they are not used to compare different damage types (validation plan in 3.3).

2.4 System Flowchart

The operating structure that links these modules as software is shown in the system flowchart below. Acquisition → preprocessing and feature extraction → environmental compensation → anomaly scoring is the continuous path repeated for every analysis window; only when the anomaly score exceeds the threshold does the flow branch into band-contribution analysis and the precision-inspection alert. Each block corresponds, in order, to steps 1-6 of 2.3.



[Figure 3: Flowchart of the bridge vibration anomaly detection system]

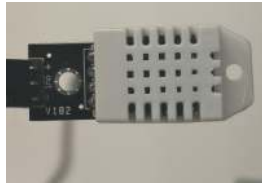
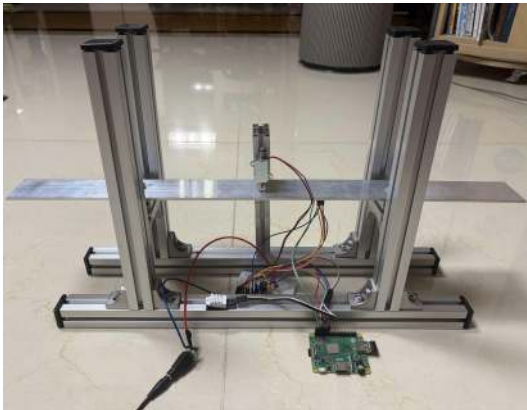
2.5 Development Process

Step	Work	Output
Step 1 (06/03)	Checked sensor links (ADXL355Z, DHT22, Pi 3A+); built the foam-board model and chose sensor positions	Connection test results, mounting photos
Step 2 (06/03-06/04)	Collected normal (0-0) and mass-condition (1-0, 1-1) data (notation in 3.1)	Raw CSV per condition
Step 3 (06/04)	Collected environmental conditions (outdoor temperature excluded, see 3.2); organized 9 conditions	9-condition dataset (baseline, humidity, temp+humidity)
Step 4 (06/04-06/05)	FFT/STFT feature extraction; Autoencoder training and anomaly scoring	Feature dataset, detection results
Step 5 (06/05-06/12)	Band-contribution analysis; stage-1 results summarized	Contribution plots, stage-1 report
Step 6 (06/17-07/08)	Purchased and sourced stage-2 parts (aluminum model, tapper)	All parts
Step 7 (07/11-07/12)	Assembled the model (rail-piers-beams-deck); tapper circuit and Pi 5 real-time dashboard implemented	Stage-2 model and build photos
Step 8 (07/13-07/18)	Normal baseline collection and 27-condition matrix experiment	CSV per condition, tap log
Step 9 (07/19-07/26)	Stage-2 analysis, model comparison, report update	Result tables, plots, final report

2.6 Components Built and Key Implementation

Hardware build

In stage 1, an ADXL355Z-DHT22-Raspberry Pi acquisition setup was built on a foam-board model about 55 cm long to verify the concept of temperature/humidity-compensated anomaly detection. Because damping and assembly variation in the foam board made the resonance peaks broad, stage 2 extended the same sensing and compensation setup to an aluminum multi-span model.

 Foam-board bridge model for stage 1 (about 55 cm long)	 ADXL355Z 3-axis MEMS accelerometer	 DHT22 temperature/humidity sensor
 Aluminum bridge model for stage 2 (about 60 cm long)	 Solenoid auto-tapper	
 Raspberry Pi 5 real-time dashboard		

2.7 Validation on Full-Scale openLAB Data

The proposed pipeline was applied to the openLAB dataset [8] published in TU Dresden's research data repository (OPARA). The structure is a full-scale, three-span prestressed concrete bridge about 45 m long; acceleration, air temperature and humidity were measured together for about nine months, February to October 2024, in the undamaged reference state, so seasonal and daily temperature variation accumulates in the data, and because the whole period is undamaged, every threshold exceedance can be interpreted as a false alarm, allowing the change in false-alarm rate (FAR) due to compensation to be quantified. openLAB acceleration responses are event-triggered records saved each time a vibration event is detected, and each event record is used as one analysis window. Vibration features were extracted as in 2.3 and linked to air temperature and humidity by time; the normal samples were then split in time order into 263 for training and 263 for evaluation, and three models — uncompensated AE, temperature-compensated AE and temperature/humidity-compensated AE — were compared under identical conditions, with the threshold set at the 99th percentile of the training reconstruction error. The results are shown in the table below.

Model	Training samples	Evaluation samples	Anomaly rate, training	False-alarm rate, evaluation
Uncompensated AE	263	263	0.011	0.019
Temperature-compensated AE	263	263	0.011	0.015
Temp/humidity-compensated AE	263	263	0.011	0.015

All three models had a training anomaly rate of 0.011, consistent with the 99th-percentile threshold, and on the evaluation period, which contains only undamaged data, the false-alarm rate was 1.90% for the uncompensated AE and 1.52% for both compensated AEs. The 0.38-point reduction (5 → 4 false alarms) is a difference of a single alarm and is hard to treat as statistical evidence on its own, but the direction — fewer false alarms after compensation on nine months of full-scale data with accumulated seasonal and daily variation — is consistent with the model experiments (3.4). That adding humidity brought no further reduction suggests the variation is explained mainly by temperature, in line with prior work [2]; the effect of humidity compensation itself is confirmed in our own model experiment, where the false-alarm rate in the humidity-change segment fell from 17.5% to 9.1%. Variation in excitation source and amplitude due to event-triggered recording is common to all three models, so it does not affect the interpretation of differences in false-alarm rate with and without compensation.

3 Testing

3.1 Test Purpose and Experimental Design

The tests have two purposes. First, to examine with low-cost sensors (2.2) and model bridges how changes in structural condition affect the vibration features and anomaly scores: stage 1 verifies that the pipeline works, and stage 2 verifies discrimination across 27 conditions. Second, to verify the reduction in false alarms from temperature/humidity compensation; the full-scale comparison is handled by openLAB (2.7), and our own experiments check that the same trend of improvement is reproduced.

Design of the weight-based mass-distribution experiment

Instead of real damage, stage 1 applies a non-destructive change in mass distribution of 100 g per point (ten 10 g weights). The load points are placed at L/4 and 3L/4, away from the sensor at mid-span, and since a single load on the right is nearly symmetric to one on the left, it is represented by 1-0. The conditions are written 0-0 (normal), 1-0 (left) and 1-1 (both sides).

Stage-2 design — aluminum multi-span model and a 27-condition matrix

The factors below are run as a matrix of 3 environment states × 3 mass levels × 3 bolt levels = 27 conditions. Because the heater changes temperature and humidity at the same time, temperature alone cannot be changed; the environment therefore has three states: no change (Tx-Hx), humidity change (Tx-Ho) and temperature/humidity change (To-Ho). This design yields interaction data between environment (T·H) and simulated damage (M·B), and tests the value of temperature/humidity compensation, which should classify environmental change as normal and damage as anomalous.

Factor (code)	Levels	How applied
Environment (T·H)	No change / humidity / temp + humidity (3 levels)	No change = Tx-Hx (no environmental input). Humidity change = Tx-Ho (opaque box + humidifier for high humidity, temperature held). Temp+humidity change = To-Ho (heater raises temperature, humidity falls with it). For reproducibility, heater distance, power, heating time and humidifier run time are logged; analysis uses DHT22 measurements.
Mass change (M)	0 / 500 / 1,000 g (3 levels)	Using two marks at x = 150 mm and 450 mm, M500 applies 500 g on one side (asymmetric) and M1000 applies 500 g on each side (symmetric). Made up of the available 50 g weights ×20, stacked in the same order and orientation every time
Bolt loosening (B)	0 / 1 / 2 bolts (3 levels)	Pre-selected deck-beam through-bolts are loosened by 1/2 turn from fully tightened. 1 bolt = BL1 (left tower, asymmetric boundary damage); 2 bolts = BL1·BL2 (left and right towers, symmetric boundary damage)

Loosening is limited to the deck-beam through-bolts, which act directly on the deck's boundary conditions, and B1 is a subset of B2 so that boundary stiffness is lowered step by step. Beam-pier and base-pier bolts are not loosened. After the bolt conditions, the original tightening is restored to check that the normal anomaly score returns to baseline. The normal class is defined as the three M0-B0 cells (three environment states), so that a state in which only the environment changed is learned as normal; because these three cells are used for training and thresholding, they are collected longer than the other conditions. The runs are grouped into three environment blocks, each containing the 9 mass $\times$ bolt conditions. Each condition receives 10 warm-up taps (not recorded) followed by 35 recorded taps; each of the three normal cells receives 180 recorded taps. The normal cells are split in time order into 120 training, 30 threshold and 30 evaluation windows, giving 360 training, 90 threshold and 90 evaluation windows in total.

Data acquisition and storage format

The acquisition program and its logging intervals are as in 2.2. Each raw CSV has columns for timestamp (ns and Unix seconds), DHT22 temperature and relative humidity, ADXL355Z internal temperature and 3-axis acceleration in g, and conditions are identified by the condition code T{x,o}-H{x,o}-M{0,500,1000}-B{0,1,2} in the file name together with repetition metadata. In analysis, z-axis features (log RMS, log p2p, peak frequency, log peak power, log total energy and band-energy ratios) are computed for each tap window and linked to the window's mean temperature/humidity and condition, and the peak frequency is used as the estimate of the first natural frequency.

3.2 Test Method

Testing is divided into analysis of the public openLAB data and our own model-bridge experiments. Because openLAB is undamaged throughout, every threshold exceedance can be treated as a false alarm, so it is used to compare false-alarm rates across compensation methods (process and results in 2.7). Our own experiments verify low-cost sensor acquisition, detection of mass change and reproduction of compensation. The acquisition and logging setup is the same as in 2.2 and 3.1 for both stages; only the differences between experiments are described below.

In stage 1, with the sensor layout of 2.2, 0-0, 1-0 and 1-1 were excited manually and recorded for at least 20-30 seconds each, repeated three or more times where possible. Because of reliability problems with outdoor temperature, the environmental conditions were reduced from the original 12 (3 masses $\times$ 4 environments) to 9 (3 masses $\times$ 3 environments: baseline, humidity-centered change, and simultaneous temperature/humidity change). The analysis window is fixed at 4 seconds. The compensation and anomaly-detection models are trained only on normal 0-0 data (three environmental conditions), and the rest is used for evaluation. The threshold — the 99th percentile of the normal reconstruction error — is applied identically to the three models (uncompensated, temperature, temperature/humidity), all of which use the PCA reconstruction of 2.3 (equivalent to a linear AE).

The key change in stage 2 is replacing manual excitation with automatic taps from the solenoid auto-tapper, to narrow the scatter of the normal baseline. Tap times and acceleration are synchronized on the same clock, and only the window right after each tap is analyzed.

Data are collected at 500 Hz sampling with a ± 8 g range, and the analysis window is fixed for all conditions at 2.0 s (1,000 samples) starting at $t_0+0.05$ s to exclude the impact. The spectrum is estimated with Welch's method, and the analysis bands are reset to 0-40, 40-80 and 80-125 Hz to match the measured first natural frequency, differing from stage 1's 0-12, 12-25 and 25-50 Hz. The features are eight z-axis quantities (log RMS, log p2p, peak frequency, log peak power, log total energy and three band-energy ratios); amplitudes are log-scaled and band energies normalized as ratios so that changes in spectral structure, not tap strength, stand out.

Training, thresholding and evaluation are kept separate to prevent data leakage. Training uses 360 normal windows; the threshold is the 99th percentile of the reconstruction error on 90 dedicated windows not used in training; evaluation uses 90 normal and 840 damage windows (split described in 3.1). The model is fixed as a nonlinear Autoencoder with an 8-3-8 hidden structure (tanh), only the environmental inputs are varied (uncompensated, temperature, temperature/humidity), and runs are repeated with 5 random seeds to check stability. The real-time decision setup is as in 2.2, and training and evaluation are done on the saved CSVs.

3.3 Expected Results

This section describes the results that changes in structural condition (mass distribution, bolt loosening) and temperature/humidity compensation are expected to have on the vibration features and anomaly scores, together with the plan for verifying them.

In stage 1, added mass is expected to lower the natural frequency, so the peak frequency should fall under the weight conditions, while in the asymmetric 1-0 changes in amplitude and band energy are expected before any frequency shift. Because indoor temperature/humidity changes are smaller than openLAB's seasonal change, the compensation effect may also be small, so the criterion is whether the trend of improvement (fewer false alarms) is reproduced rather than its absolute size. In stage 2, the added mass exceeds the deck mass (about 243 g), but because the weights sit near the supports, the effective mass on the first mode is smaller than it appears. The first natural frequency should therefore fall clearly but not by as much as the mass ratio, and the actual size will be fixed by measurement after assembly. The focus of observation is ordering rather than detection: we check whether the anomaly score and ratio increase monotonically as conditions within the same damage type become more severe (M0→M500→M1000, B0→B1→B2), which must hold for the anomaly score to serve as a severity indicator for inspection priority. Comparisons between different types are excluded from validation because their physical references differ and no correct answer can be defined; in operation they are presented only as a relative ranking of departure from normal. Bolt loosening is expected to bring a small drop in natural frequency accompanied by secondary changes such as harmonics and wider peaks. In short, we test the hypothesis that mass is distinguished by the size of the frequency shift and bolts by the combination of accompanying features, and also check whether the reverse shift of 1-0, interpreted as scatter in stage 1, disappears with automatic excitation and whether the compensation effect is reproduced in the three normal cells where only the environment changed.

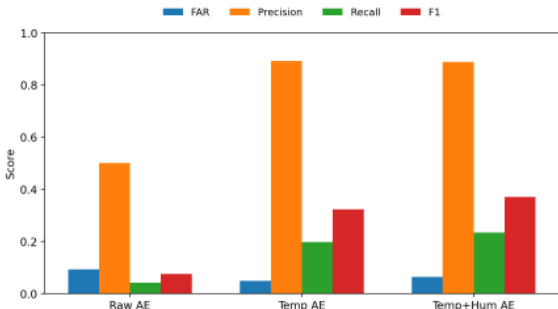
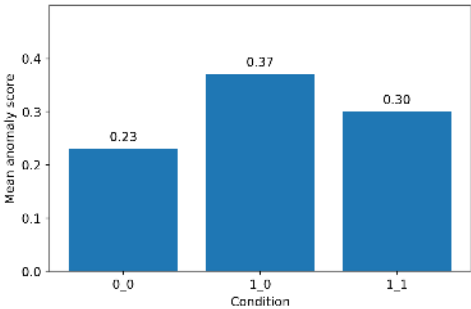
3.4 Actual Results

In the preliminary test the first peak was observed at about 22.6 Hz, so the base analysis bands were reset to 0-12, 12-25 and 25-50 Hz. The mass change of 1-0 was detected by the multi-feature anomaly score, raising the exceedance rate to 29.7%. In the humidity-change segment the false-alarm rate fell from 17.5% uncompensated to 9.1% with temperature/humidity compensation. This 8.4-point reduction is actually larger than openLAB's 0.38 points, because humidity can be changed strongly indoors, so the false-alarm rate before compensation was high and there was correspondingly more room to reduce it. The stage-1 results are presented in two tables and Figures 4 and 5; the first table aggregates by condition and the second gives per-model metrics over the whole evaluation segment, so their populations differ and they are not directly comparable. In Figure 4, check whether the mean anomaly score of the weight conditions (1-0, 1-1) rises above normal (0-0); in Figure 5, check whether the metrics improve as compensation steps are added.

Condition ID	Left load	Right load	Mean f	Change in f	Mean anomaly score
0-0	0g	0g	22.6 Hz	+0.0%	0.23
1-0	100g	0g	22.8 Hz	+0.8%	0.37
1-1	100g	100g	20.6 Hz	-8.9%	0.30

Model	Environmental inputs	False Alarm Rate	Precision	Recall	F1-score
Uncompensated AE	None	0.092	0.500	0.041	0.075
Temperature-compensated AE	Temperature T	0.048	0.892	0.197	0.323
Temp/humidity-compensated AE	Temperature T + relative humidity RH	0.063	0.888	0.234	0.370

Metrics in the table are over the whole evaluation segment; the 17.5%→9.1% in the text is the false-alarm rate limited to the humidity-change segment.



[Figure 4: Mean anomaly score by condition]

[Figure 5: Performance metrics by compensation step]

Stage-2 results

With the temperature/humidity-compensated model, all 840 damage windows exceeded the threshold (recall 100%), and there were 4 false alarms among the 90 normal evaluation windows (4.4%). All four occurred in the simultaneous temperature/humidity (To·Ho) block; the 4/30 (13.3%) in the table below is the same four alarms counted against that block's 30 windows. All 1,380 windows of the 27 conditions were valid, none discarded. The measured normal first natural frequency is 101.0 Hz, and the anomaly threshold is a reconstruction error of 5.6064. The mean anomaly scores in the table are absolute values on the same scale as this threshold, while the ratios (2.2) in the text and Figure 6 are those values divided by the threshold. Each damage group in the table sums the three environment blocks (105 windows each), and the distribution over all 27 cells is shown in Figure 6. The peak shifts in the table are far larger than the gradual drop expected in 3.3 (e.g., -64.4%), and there are even reversals where more severe damage gives a smaller shift (M1000·B1 -19.8%). This does not mean the natural frequency actually fell that far. A structure has several vibration modes at once, and the peak frequency reads only the single tallest peak; when damage changes the strength of each mode, the tallest peak jumps from the original 101 Hz mode to a lower mode (mode switching). The value therefore jumps instead of falling continuously, and since which mode becomes tallest is not monotonic in the size of the damage, reversals are natural (detailed interpretation in 3.5).

Condition group	Median peak frequency	Peak shift (vs 101 Hz)	Mean anomaly score	Exceedance rate
Normal baseline (Tx·Hx·M0·B0)	101.0 Hz	+0.0%	0.03	0/30 (0%)
Humidity change (Tx·Ho·M0·B0)	101.0 Hz	+0.0%	0.07	0/30 (0%)
Temp + humidity (To·Ho·M0·B0)	90.0 Hz	-10.9%	4.13	4/30 (13.3%)
M500·B0	36.0 Hz	-64.4%	33.2	105/105 (100%)
M1000·B0	36.0 Hz	-64.4%	128.4	105/105 (100%)
M0·B1	64.0 Hz	-36.6%	264.7	105/105 (100%)
M0·B2	40.0 Hz	-60.4%	370.3	105/105 (100%)
M500·B1	44.0 Hz	-56.4%	75.4	105/105 (100%)
M1000·B1	81.0 Hz	-19.8%	108.3	105/105 (100%)
M500·B2	66.0 Hz	-34.7%	307.1	105/105 (100%)
M1000·B2	53.0 Hz	-47.5%	323.4	105/105 (100%)

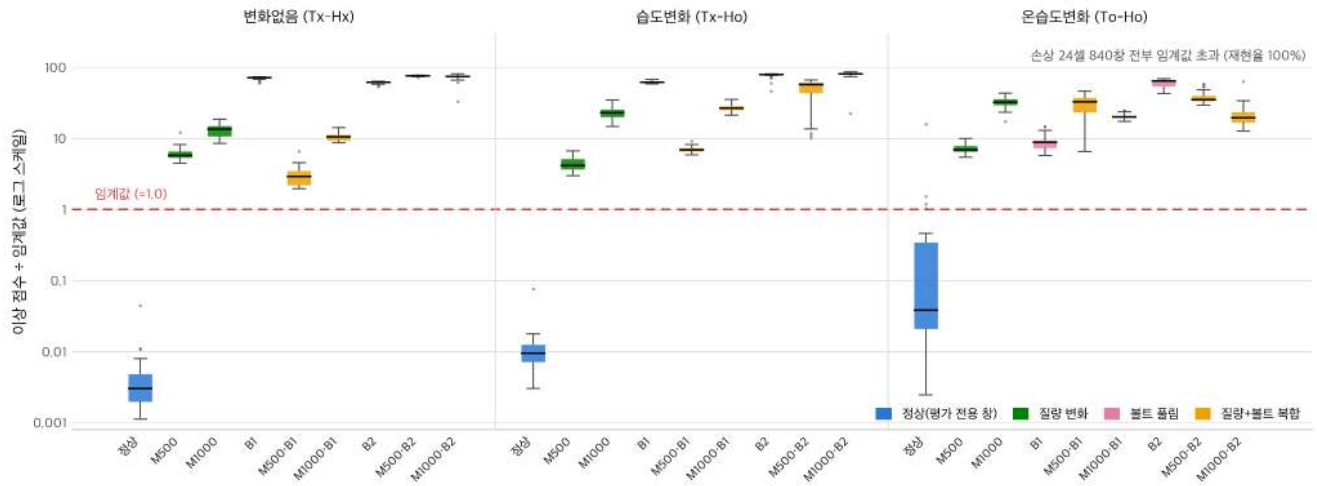
Model	False Alarm Rate	Precision	Recall	F1-score
Uncompensated AE (no environmental input)	0.022	0.998	0.993	0.995
Temperature-compensated AE (T residual)	0.033	0.996	1.000	0.998
Temp/humidity-compensated AE (T·RH residual)	0.044	0.995	1.000	0.998

Over 5 repeated seeds, both compensated models kept 100% recall for every seed, while the uncompensated AE varied between 96.2% and 99.3% (interpretation in 3.5). The ROC-AUC of the three models is 0.994-0.996.

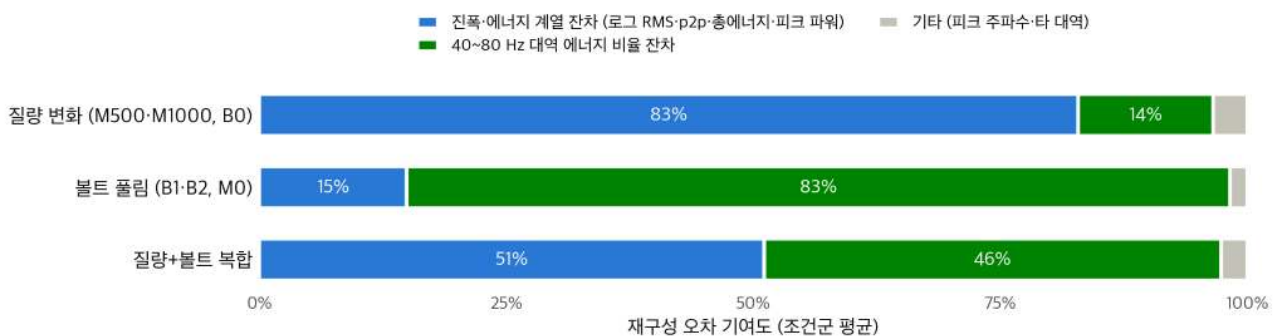
In stage 2 the FAR reduction seen in stage 1 was not reproduced; the benefit of compensation appeared as more stable damage detection. Even the uncompensated AE had a low false-alarm rate, 0-6.7%, on normal evaluation windows where only the environment changed. The table below shows both metrics (interpretation in 3.5).

Metric	Uncompensated AE	Temp/humidity-compensated AE
FAR, humidity block (Tx·Ho)	0.0% (0/30)	0.0% (0/30)
FAR, temp+humidity block (To·Ho)	6.7% (2/30)	13.3% (4/30)
Damage recall (840 windows, range over 5 seeds)	96.2~99.3%	100% (all seeds)

Environmental false alarms were equally low for both models, so the real difference is in the damage-recall row. The 2 versus 4 in To-Ho is the same level, since the binomial confidence intervals over 30 windows overlap, and 3 of the 4 compensated false alarms are borderline exceedances at 1.0-1.5× the threshold.



[Figure 6: Distribution of anomaly scores over the 27 stage-2 conditions (ratio to threshold, log scale) — all 840 damage windows exceed the threshold]



[Figure 7: Feature contributions to reconstruction error by damage type — mass, bolt and combined profiles]

3.5 Analysis of Results

The stage-1 results matched the expected physics. Under loading on both sides (1-1), the mean peak frequency fell by 8.9%, exactly as expected from the natural frequency falling as mass increases ($f = (1/2\pi)\sqrt{k/m}$); with the same stiffness k , a larger mass m lowers f). The main contributing band also moved from 0-12 Hz to 12-25 Hz, where the first resonance lies, supporting the same interpretation. Under loading on one side (1-0) the peak

rose by 0.8%, but this is within the scatter of manual tapping and is not a meaningful change. Instead, the amplitude and band features responded, and the exceedance rate jumped from 4.5% (normal 0-0 windows) to 29.7% (1-0). This confirms the design assumption that a small asymmetric mass change must be caught by an anomaly score combining several features, not by a single frequency. The remaining problems were that damping in the foam board blurred the resonance peak, burying small frequency changes in the peak width, and that manual tapping varied in force and timing every time, enlarging the scatter of normal measurements even without damage. In unsupervised detection, which flags anything outside the normal scatter, a wide normal scatter means correspondingly small damage is missed, so these two problems became the improvement targets of stage 2.

The stage-2 results show four things.

First, standardizing the excitation with a machine fixed the normal baseline. After the auto-tapper was introduced, the peaks of the 60 room-temperature normal evaluation windows converged on a single value, 101.0 Hz, with a standard deviation of 0.0 Hz. This means the variation in force and timing of manual tapping, a problem in stage 1, disappeared, and the 0.8% reverse peak shift attributed to that variation in stage 1 did not reappear. Second, the 27 conditions were separated almost completely. The figures are as in the table in 3.4; the key point is the gap between normal and damage. The median ratio was 0.003-0.038× for normal windows and 5.8-73.7× for damage windows, a gap of more than a hundredfold (Figure 6). The tails did not separate completely, however: one normal window in the heated block spiked to 15.9×, and the weakest damage window reached only 2.0×. In actual operation, therefore, decisions are based on the exceedance rate over consecutive windows rather than a single window. The severity ordering was clean for mass: the ratio grew monotonically from 5.8× for M500 to 22.6× for M1000 in all three blocks. For bolts, loosening even one bolt saturated the score at tens of times the threshold, so the ordering held only in the low-severity range, and in the no-change block B1's 71.5× even exceeded B2's

61.5×, a reversal.

Third, the composition of reconstruction-error contributions points to the type of damage (Figure 7). In windows with loosened bolts, the 40-80 Hz band-ratio residual accounted for 76-90% of the reconstruction error; in windows with added mass, amplitude/energy features such as log RMS accounted for 73-80%. Read in reverse, this becomes an operating guideline: if the 40-80 Hz band ratio dominates an alarm's contributions, suspect damage to boundary conditions such as bolted joints first; if the amplitude/energy features dominate, suspect a change in load (mass) and start the inspection there. That the two groups appeared side by side at 51% and 46% in combined mass + bolt windows supports this. It is not a diagnosis that fixes location and type, but it sets the starting point for the follow-up precision inspection. Meanwhile, the peak frequency swung far more than expected for both mass and bolts because of mode switching (3.4) (bolt B1 -36.6%, B2 -60.4%), showing that the stage-1 conclusion — detect by combining several features rather than relying on a single peak — remained valid in stage 2.

Fourth, the role of temperature/humidity compensation changed from stage 1. Thanks to stepwise heating and continuous humidification, the normal training data already covered 22.4-39.9 °C and 48.9-79.0% humidity, environmental change rarely caused false alarms even without compensation, and there was no room for a reduction like that in stage 1. Instead, the benefit of compensation appeared as more stable detection: when training was repeated with 5 seeds, the recall of the uncompensated model fluctuated between 96.2% and 99.3%, whereas the compensated model reached 100% with all five.

The largest share of the performance gain came from feature normalization. Because amplitudes differed by up to about 40× between conditions, standardization with stage 1's raw features was dominated by amplitude, giving a recall of only 21.3% (in a reanalysis applying the raw features directly to the stage-2 data) and 0% for mass windows, whereas switching to log and ratio normalization raised it to 100% on the same data.

The remaining 4 false alarms are all windows near 39.9 °C, the hottest point of the heating — a limitation of approximating the structure's temperature with air temperature and humidity alone; improvements are discussed in 4.2. The improvements from stage 1 to stage 2 are summarized in the table below.

Item	Stage 1	Stage 2
Model · excitation	Foam-board model (55 cm) · manual tapping	Aluminum multi-span model (60 cm) · solenoid auto-tapper (40 ms, 5 s period)
Conditions · window	9 conditions · 4 s window (200 Hz)	27 conditions, 1,380 windows · 2.0 s window (500 Hz)
Normal peak repeatability	22.6 Hz, varies by window (manual scatter)	101.0 Hz, SD 0.0 Hz over 60 room-temp windows
Damage detection	Window-level exceedance rate 18.8~29.7% (1-1·1-0)	Window-level recall 100% (840/840)
False alarms	Humidity segment 9.1% (compensated)	All evaluation windows 4.4% (4/90)
Model · features	PCA reconstruction (linear equiv.) · raw-scale features	Nonlinear AE (8-3-8) · log/ratio-normalized features

4 Interpretation & Conclusion

4.1 Conclusion

Using only low-cost commercial parts (ADXL355Z, DHT22, Raspberry Pi), this system demonstrated the target performance of first-stage screening, prior to precision inspection, on the 27-condition matrix of a laboratory model. The temperature/humidity-compensated model caught all 840 damage windows and raised only 4 false alarms among 90 normal evaluation windows, recording window-level recall of 100%, a false-alarm rate of 4.4%, F1 0.998 and ROC-AUC 0.996. Whereas in stage 1 it was a system that was usually right when it alarmed but often missed damage (precision 0.888, recall 0.234), in stage 2, after standardized excitation and a redesign of feature normalization, it became a system that alarms accurately without misses.

The value of temperature/humidity compensation appeared in three different forms. Where training had not seen enough environmental variation, it reduced false alarms directly (stage 1: 17.5%→9.1%; openLAB: 1.90%→1.52%); in stage 2, where environmental coverage was secured, it appeared as detection stability, keeping recall at 100% regardless of the random seed. On top of this, contribution analysis of the reconstruction error changes the quality of an alarm. With a system that gives only an anomaly signal, an inspector cannot know what to examine, but this system also shows which features increased the error. In stage 2, the 40-80 Hz band-ratio residual accounted for 76-90% of the contribution for bolt loosening and the amplitude/energy residuals for 73-80% for mass change, so the alarm alone was enough to narrow the direction to a boundary anomaly or a load change. It does not fix the location or type of damage, but it provides a practical clue as to where the follow-up precision inspection should start. The within-type severity ordering held in every block for mass, providing grounds for using it as an inspection-priority indicator, and for bolts it held only in the low-severity range B0→B1, before the score saturated. On the model side, a nonlinear Autoencoder with an 8-3-8 hidden structure, built on log/ratio-normalized features, replaced stage 1's linear PCA implementation, but the dominant factor in the performance gain was the feature representation rather than model capacity. All of these figures, however, come from short-term indoor model experiments. Traffic loads, seasonal cycles and long-term drift on a real bridge have not yet been experienced, so any judgment on practical use must wait until the field validation in 4.2 has been passed.

4.2 Future Improvements

Future improvement focuses on a single goal: field validation on a real bridge. What could be checked on a model was checked in stage 2, so the remaining question is whether the same performance holds in a real operating environment. For the sensor's own measurement performance we rely on prior work [1], which validated modal measurement of real structures with the same ADXL355 family; instead, short measurements at several candidate bridges will compare the signal-to-noise ratio (SNR) of ambient vibration, and the bridge with the sharpest resonance peak will be chosen. Candidates are limited to pedestrian overpasses (Class 3 scale) that recently passed a precision inspection, so the inspection record supports the assumption of a normal state at the start of measurement; a single node with a waterproof enclosure, power and communications is installed under the center of the main span, and excitation switches to the ambient vibration produced by pedestrians and wind.

Next is quantifying the compensation effect in the real environment. After training the regression, Autoencoder and threshold on site with the first week of normal data following installation, two to three weeks of undamaged continuous monitoring will compare the false-alarm rates of the uncompensated and compensated models side by side, measuring with our own system's data how much temperature/humidity compensation reduces false alarms under the daily temperature range and weather changes of a real bridge. Upper limits for the window-level false-alarm rate and for the number of consecutive-window false alarms will be declared before monitoring begins; procedures for monthly retraining and threshold updates and the measurement of baseline drift on sensor remounting will be established in this period, and the limitation of approximating structural temperature will be addressed by adding a deck surface-temperature sensor. Validation over all four seasons is a long-term task to be continued through permanent operation of the node.

Finally, detection power will be checked. Because damage cannot be inflicted on a real bridge, reversible controlled disturbances — a known mass placed at a designated position with the cooperation of the managing authority — will be inserted midway through and at the end of monitoring. The size of the disturbance will be calculated in advance to match the change in natural frequency that real damage produces according to the literature (a few percent), and two to three load levels will be used to check whether the stage-2 severity ordering is reproduced. Passing these three steps would provide the validation basis for moving, via comparison with periodic inspections, multi-node expansion and a review of damage localization, toward a continuous screening tool for small and medium bridges.

References

- [1] Caballero-Russi, D., Ortiz, A. R., Guzmán, A., & Canchila, C., “Design and Validation of a Low-Cost Structural Health Monitoring System for Dynamic Characterization of Structures,” Applied Sciences, 12(6), 2807, 2022.
- [2] Peeters, B., & De Roeck, G., “One-year monitoring of the Z24-Bridge: environmental effects versus damage events,” Earthquake Engineering & Structural Dynamics, 30(2), 149-171, 2001.
- [3] Z24 Bridge Benchmark documentation, KU Leuven, Leuven, Belgium.
- [4] Korea Research Institute of Standards and Science (KRISS), distributed fiber-optic sensor technology based on BOCDA (Brillouin Optical Correlation Domain Analysis).
- [5] Giglioni, V. et al., “Deep Autoencoders for Unsupervised Damage Detection with Application to the Z24 Benchmark Bridge,” IMAC, Springer, 2022.
- [6] Pakzad, S. S., & Masoodi, A. R., “Early damage detection in bridges using a variational autoencoder-based hybrid unsupervised learning framework,” Scientific Reports, 16, 1389, 2026. DOI: 10.1038/s41598-025-31115-w
- [7] Aosong Electronics, DHT22 (AM2302) temperature/humidity sensor datasheet.
- [8] Jansen, A., Richter, B., Röder, R., Herbers, M., & Marx, S., "Monitoring Data of the openLAB Research Bridge (2024-02-01 to 2024-10-31) and building information," OPARA - Open Access Repository and Archive, TU Dresden, 2025. DOI: 10.25532/OPARA-660.

List of Materials

No.	Type	Label in text	Description (keywords)	Notes (source, etc.)
1	Photo	Material 1	Jeongja Bridge collapse site photo	Safety Journal report (2023.4.7), https://www.anjunj.com/news/articleView.html?idxno=36589
2	Photo	Material 2	Fern Hollow Bridge collapse photo	Taken by the NTSB (U.S. federal government work, public domain), posted on Wikimedia Commons
3	Chart	Material 3	Share of Class 3 bridges among all bridges in Korea	Drawn by the authors from the Korea Authority of Land & Infrastructure Safety's 2025 facility statistics
4	Table	Material 4	Inspection cycles and targets by facility grade	Enforcement Decree of the Special Act on the Safety and Maintenance of Facilities, Annex 3 (amended 2025.12.4); drawn by the authors
5	Diagram · plot	Figures 1-7	Layout drawing, Autoencoder architecture, system flowchart, experimental result plots	Made by the authors
6	Photo	All photos	Model bridges, finished module, stage-2 model build photos	Taken by the authors

7	공개 데이터	데이터셋	openLAB(TU Dresden, OPARA — 2.7 분석에 사용), Z24 Bridge Benchmark(KU Leuven — 선행 연구 비교 참조)	공개 데이터, 참고문헌 3·8
8	오픈소스 SW	-	Python, NumPy, pandas, scikit-learn, Matplotlib (수집·분석·시각화)	각 공식 배포본(PSF·BSD 라이선스)

소스코드

Sentry.py

```

from __future__ import annotations

import argparse
import io
import json
import math
import socket
import subprocess
import sys
import threading
import time
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from flask import Flask, jsonify
from scipy.signal import welch
from sklearn.linear_model import LinearRegression
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler

HTML = r'''<!doctype html>
<html lang="ko">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>SENTRY</title>
<style>
body{margin:0;background:#10141b;color:#eef2f7;font-family:Arial,sans-serif}.wrap{
max-width:1200px;margin:auto;padding:20px}.top{display:flex;justify-content:space-
between;align-items:center;margin-bottom:16px}.title{font-size:28px;font-weight:70
0}.grid{display:grid;grid-template-columns:repeat(6,1fr);gap:12px}.card{background
:#1a202b;border-radius:14px;padding:16px;box-shadow:0 6px 18px
#0004}.wide{grid-column:span 3}.full{grid-column:span
6}.label{font-size:13px;color:#9ca8b8}.value{font-size:28px;font-weight:700;margin
-top:6px}.status{padding:10px
18px;border-radius:999px;font-weight:700}.normal{background:#176b45}.warning{backg
round:#916b12}.anomaly{background:#9d2c2c}.waiting{background:#465164}canvas{width
:100%;height:250px;background:#111722;border-radius:10px}.bands{display:flex;heigh
t:26px;border-radius:8px;overflow:hidden;margin-top:12px}.band{display:flex;align-
items:center;justify-content:center;min-width:0;font-size:12px}.b1{background:#3f8
2d3}.b2{background:#c98b2e}.b3{background:#bd4f59}.small{font-size:12px;color:#9ca
8b8;margin-top:8px}@media(max-width:800px){.grid{grid-template-columns:repeat(2,1f
r)}.wide,.full{grid-column:span 2}}
</style>
</head>

```

```

<body>
<div class="wrap">
<div class="top"><div><div class="title">SENTRY 실시간 대시보드</div><div
id="connection" class="small">서버 연결 확인 중</div></div><div id="status"
class="status waiting">대기 중</div></div>
<div class="grid">
<div class="card"><div class="label">이상 점수 / 임계값</div><div id="score"
class="value">-</div></div>
<div class="card"><div class="label">피크 주파수</div><div id="peak"
class="value">-</div></div>
<div class="card"><div class="label">RMS</div><div id="rms"
class="value">-</div></div>
<div class="card"><div class="label">온도</div><div id="temp"
class="value">-</div></div>
<div class="card"><div class="label">습도</div><div id="hum"
class="value">-</div></div>
<div class="card"><div class="label">누적 이상 후보</div><div id="detection"
class="value">-</div></div>
<div class="card wide"><div class="label">최근 z축 파형</div><canvas
id="wave"></canvas></div>
<div class="card wide"><div class="label">최근 타격 주파수 스펙트럼</div><canvas
id="spec"></canvas></div>
<div class="card full"><div class="label">대역별 재구성 오차 기여도</div><div
class="bands"><div id="b1" class="band b1"></div><div id="b2" class="band
b2"></div><div id="b3" class="band b3"></div></div><div id="info"
class="small">-</div></div>
</div>
</div>
<script>
function line(canvas,x,y){const
c=document.getElementById(canvas),d=devicePixelRatio||1,w=c.clientWidth,h=c.clientHeight;c.width=w*d;c.height=h*d;const
g=c.getContext('2d');g.scale(d,d);g.clearRect(0,0,w,h);g.strokeStyle='#2c3544';g.beginPath();for(let i=1;i<5;i++){let
yy=h*i/5;g.moveTo(0,yy);g.lineTo(w,yy)}g.stroke();if(!y||y.length<2)return;let
ymin=Math.min(...y),ymax=Math.max(...y);if(ymax===ymin){ymax+=1;ymin-=1}g.strokeStyle='#73b7ff';g.lineWidth=2;g.beginPath();for(let i=0;i<y.length;i++){let
px=i/(y.length-1)*w,py=h-(y[i]-ymin)/(ymax-ymin)*h;if(i===0)g.moveTo(px,py);else
g.lineTo(px,py)}g.stroke()}
function ratioText(v){if(v===null)return '-';if(v>=1000)return
v.toExponential(2)+'x';return v.toFixed(2)+'x'}
async function update(){const conn=document.getElementById('connection');try{const
r=await fetch('/api',{cache:'no-store'});if(!r.ok)throw new Error('api');const
s=await r.json();const
st=document.getElementById('status');st.textContent=s.status_text;st.className='st
atus
'+s.status_class;document.getElementById('score').textContent=ratioText(s.score_ra
tio);document.getElementById('peak').textContent=s.peak_hz===null?'-':s.peak_hz.toF
ixed(2)+'
Hz';document.getElementById('rms').textContent=s.rms===null?'-':s.rms.toFixed(4)+'
g';document.getElementById('temp').textContent=s.temp_c===null?'-':s.temp_c.toF
ixed(1)+'
°C';document.getElementById('hum').textContent=s.humidity===null?'-':s.humidity.toF
ixed(1)+'
%';document.getElementById('detection').textContent=s.evaluated_count?String(s.ano
maly_count)+' /
'+String(s.evaluated_count):'-';line('wave',s.wave_x,s.wave_y);line('spec',s.freq_
x,s.freq_y);const b=s.bands||[0,0,0],ids=['b1','b2','b3'];for(let
i=0;i<3;i++){const
e=document.getElementById(ids[i]);e.style.width=(b[i]*100)+'%';e.textContent=b[i]>
0.08?Math.round(b[i]*100)+'%':'-' }document.getElementById('info').textContent=s.inf
o||'-';if(s.updated_at===null){conn.textContent='서버 연결됨 · 데이터 대기'}else{const
age=Math.max(0,Date.now()/1000-s.updated_at);conn.textContent=age<3?'서버 연결됨 ·
실시간 수집 중': '서버 연결됨 · 마지막 데이터 '+age.toFixed(1)+'초

```

```

전'}}catch(e){conn.textContent='서버 연결 끊김';const
st=document.getElementById('status');st.textContent='연결
끊김';st.className='status waiting'}}
setInterval(update,700);update();
</script>
</body></html>'

```

```

RAW_FEATURES = ["rms", "std", "p2p", "peak_hz", "peak_power", "energy_1",
"energy_2", "energy_3"]
FEATURES = ["log_rms", "log_p2p", "peak_hz", "log_peak_power", "log_total_energy",
"band_frac_1", "band_frac_2", "band_frac_3"]
FILTER_LPF = {"0x03": 125.0, "0x04": 62.5, "0x05": 31.25}
EPS = 1e-12

```

```

def prepare_features(raw: pd.DataFrame | np.ndarray) -> np.ndarray:
    if isinstance(raw, pd.DataFrame):
        x = raw[RAW_FEATURES].to_numpy(float)
    else:
        x = np.asarray(raw, dtype=float)
        if x.ndim == 1:
            x = x.reshape(1, -1)
    total = x[:, 5] + x[:, 6] + x[:, 7] + EPS
    return np.column_stack([
        np.log10(np.maximum(x[:, 0], EPS)),
        np.log10(np.maximum(x[:, 2], EPS)),
        x[:, 3],
        np.log10(np.maximum(x[:, 4], EPS)),
        np.log10(total),
        x[:, 5] / total,
        x[:, 6] / total,
        x[:, 7] / total,
    ])

```

```

def model_features(model: dict, raw: pd.DataFrame | np.ndarray) -> np.ndarray:
    if model.get("feature_version") == "log_normalized_v1":
        return prepare_features(raw)
    if isinstance(raw, pd.DataFrame):
        return raw[RAW_FEATURES].to_numpy(float)
    x = np.asarray(raw, dtype=float)
    return x.reshape(1, -1) if x.ndim == 1 else x

```

```

def companion_paths(bridge: Path) -> tuple[Path, Path]:
    name = bridge.name
    if not name.endswith("_bridge_log.csv"):
        raise ValueError("bridge CSV 이름은 *_bridge_log.csv 형식이어야 합니다.")
    base = name[:-len("_bridge_log.csv")]
    return bridge.with_name(base + "_tap_events.csv"), bridge.with_name(base +
"_metadata.json")

```

```

def metadata_for(bridge: Path) -> dict:
    _, meta = companion_paths(bridge)
    if meta.exists():
        return json.loads(meta.read_text(encoding="utf-8"))
    return {"sample_rate_hz": 500, "range_g": 8, "filter_register_hex": "0x03"}

```

```

def limits(meta: dict) -> tuple[float, float, list[float]]:
    fs = float(meta.get("sample_rate_hz", 500))
    lpf = FILTER_LPF.get(str(meta.get("filter_register_hex", "0x03")).lower(),
min(fs / 2, 125.0))

```

```

top = min(fs / 2, lpf)
if top >= 120:
    bands = [0.0, 40.0, 80.0, top]
else:
    bands = [0.0, top / 3, top * 2 / 3, top]
return fs, top, bands

def read_taps(path: Path) -> pd.DataFrame:
    df = pd.read_csv(path)
    if "pulse_start_perf_ns" in df.columns:
        df["tap_ns"] = pd.to_numeric(df["pulse_start_perf_ns"], errors="coerce")
    elif "t_perf_ns" in df.columns:
        df["tap_ns"] = pd.to_numeric(df["t_perf_ns"], errors="coerce")
    else:
        raise ValueError("tap CSV에 pulse_start_perf_ns 또는 t_perf_ns가 필요합니다.")
    return df.dropna(subset=["tap_ns"]).reset_index(drop=True)

def extract_one(seg: pd.DataFrame, fs: float, top: float, bands: list[float]) ->
tuple[np.ndarray, float, float, np.ndarray, np.ndarray]:
    z = pd.to_numeric(seg["az_g"], errors="coerce").to_numpy(float)
    z = z[np.isfinite(z)]
    if len(z) < max(64, int(fs * 0.5)):
        raise ValueError("분석 구간의 유효 샘플이 부족합니다.")
    z = z - np.mean(z)
    rms = float(np.sqrt(np.mean(z * z)))
    std = float(np.std(z))
    p2p = float(np.ptp(z))
    nperseg = min(len(z), max(256, int(fs)))
    freq, power = welch(z, fs=fs, nperseg=nperseg)
    valid = (freq >= 1.0) & (freq <= top)
    if not np.any(valid):
        raise ValueError("유효 주파수 대역이 없습니다.")
    vf, vp = freq[valid], power[valid]
    idx = int(np.argmax(vp))
    peak_hz = float(vf[idx])
    peak_power = float(vp[idx])
    energies = []
    for lo, hi in zip(bands[:-1], bands[1:]):
        m = (freq >= lo) & (freq < hi)
        energies.append(float(np.trapezoid(power[m], freq[m])) if np.any(m) else
0.0)
    vector = np.array([rms, std, p2p, peak_hz, peak_power, *energies], dtype=float)
    return vector, peak_hz, rms, vf, vp

def extract_events(bridge: Path, delay: float, window: float) ->
tuple[pd.DataFrame, np.ndarray, dict]:
    tap_path, _ = companion_paths(bridge)
    meta = metadata_for(bridge)
    fs, top, bands = limits(meta)
    data = pd.read_csv(bridge)
    taps = read_taps(tap_path)
    t = pd.to_numeric(data["t_perf_ns"], errors="coerce").to_numpy(float)
    rows, env = [], []
    for _, tap in taps.iterrows():
        start = float(tap["tap_ns"]) + delay * 1e9
        end = start + window * 1e9
        a, b = np.searchsorted(t, [start, end])
        if b - a < max(64, int(fs * window * 0.7)):
            continue
        seg = data.iloc[a:b]
        try:

```

```

        vector, _, _, _ = extract_one(seg, fs, top, bands)
    except ValueError:
        continue
    temp = float(pd.to_numeric(seg.get("temp_c"), errors="coerce").mean()) if
"temp_c" in seg else math.nan
    hum = float(pd.to_numeric(seg.get("humidity"), errors="coerce").mean()) if
"humidity" in seg else math.nan
    rows.append(vector)
    env.append([temp, hum])
    if not rows:
        raise RuntimeError("유효한 타격 구간을 찾지 못했습니다.")
    return pd.DataFrame(rows, columns=RAW_FEATURES), np.asarray(env, dtype=float),
{"fs": fs, "top": top, "bands": bands}

def fill_env(env: np.ndarray, medians: np.ndarray | None = None) ->
tuple[np.ndarray, np.ndarray]:
    x = env.copy()
    if medians is None:
        medians = np.nanmedian(x, axis=0)
        medians = np.where(np.isfinite(medians), medians, 0.0)
    for j in range(x.shape[1]):
        x[~np.isfinite(x[:, j]), j] = medians[j]
    return x, medians

def train(args: argparse.Namespace) -> None:
    bridges = [Path(value) for value in args.bridge]
    parts = []
    file_peak_medians = []
    config = None
    requested = args.train_count + args.threshold_count + args.test_count
    for bridge in bridges:
        feat_part, env_part, part_config = extract_events(bridge, args.delay,
args.window)
        if config is None:
            config = part_config
        elif part_config != config:
            raise RuntimeError("모든 정상 파일은 같은 샘플링 주파수와 분석 대역을 사용해야
합니다.")
        if len(feat_part) < requested:
            raise RuntimeError(f"{bridge.name}: 유효 정상 타격이 {requested}회보다
적습니다.")
        parts.append((bridge, feat_part.reset_index(drop=True), env_part))
        file_peak_medians.append((bridge.name,
float(feat_part["peak_hz"].median())))

    train_raw = pd.concat([part.iloc[:args.train_count] for _, part, _ in parts],
ignore_index=True)
    threshold_raw = pd.concat([part.iloc[args.train_count:args.train_count +
args.threshold_count] for _, part, _ in parts], ignore_index=True)
    test_raw = pd.concat([part.iloc[args.train_count +
args.threshold_count:requested] for _, part, _ in parts], ignore_index=True)
    train_env_raw = np.vstack([env[:args.train_count] for _, _, env in parts])
    threshold_env_raw = np.vstack([env[args.train_count:args.train_count +
args.threshold_count] for _, _, env in parts])
    test_env_raw = np.vstack([env[args.train_count +
args.threshold_count:requested] for _, _, env in parts])

    train_feat = prepare_features(train_raw)
    threshold_feat = prepare_features(threshold_raw)
    test_feat = prepare_features(test_raw)
    train_env, medians = fill_env(train_env_raw)
    threshold_env, _ = fill_env(threshold_env_raw, medians)

```

```

test_env, _ = fill_env(test_env_raw, medians)
spread = np.nanstd(train_env, axis=0)
use_env = bool(spread[0] >= 0.2 or spread[1] >= 1.0)
env_model = None
if use_env:
    env_model = LinearRegression().fit(train_env, train_feat)
    train_feat = train_feat - env_model.predict(train_env)
    threshold_feat = threshold_feat - env_model.predict(threshold_env)
    test_feat = test_feat - env_model.predict(test_env)

scaler = StandardScaler().fit(train_feat)
x_train = scaler.transform(train_feat)
x_threshold = scaler.transform(threshold_feat)
ae = MLPRegressor(
    hidden_layer_sizes=(8, 3, 8),
    activation="tanh",
    solver="adam",
    alpha=1e-4,
    batch_size=min(32, len(x_train)),
    learning_rate_init=1e-3,
    max_iter=500,
    early_stopping=True,
    validation_fraction=0.15,
    n_iter_no_change=30,
    random_state=args.seed,
).fit(x_train, x_train)
threshold_error = np.mean((ae.predict(x_threshold) - x_threshold) ** 2, axis=1)
threshold = float(np.quantile(threshold_error, 0.99))
x_test = scaler.transform(test_feat)
test_error = np.mean((ae.predict(x_test) - x_test) ** 2, axis=1)
far = float(np.mean(test_error > threshold))
model = {
    "features": FEATURES,
    "raw_features": RAW_FEATURES,
    "feature_version": "log_normalized_v1",
    "config": config,
    "delay": args.delay,
    "window": args.window,
    "env_medians": medians,
    "env_model": env_model,
    "environment_enabled": use_env,
    "environment_variant": "temp_hum",
    "scaler": scaler,
    "autoencoder": ae,
    "threshold": threshold,
    "train_count": len(train_raw),
    "threshold_count": len(threshold_raw),
    "test_count": len(test_raw),
    "normal_test_scores": test_error.tolist(),
    "baseline_files": [str(path) for path in bridges],
}
joblib.dump(model, args.model)
print(f"모델 저장: {args.model}")
print(f"정상 파일: {len(bridges)}개")
print(f"타격 분할: 학습 {len(train_raw)}, 임계값 {len(threshold_raw)}, 정상 테스트 {len(test_raw)}")
print(f"환경 보정: {'사용' if use_env else '미사용'}")
print(f"정상 테스트 FAR: {far:.4f}")
if len(file_peak_medians) > 1:
    peak_values = np.asarray([value for _, value in file_peak_medians],
dtype=float)
    peak_gap = float(np.nanmax(peak_values) - np.nanmin(peak_values))
    peak_reference = float(np.nanmedian(peak_values))
    if peak_gap > max(5.0, peak_reference * 0.10):

```

```

        print("경고: 정상 파일 사이의 대표 피크 주파수 차이가 큼니다. 센서, 지지점, 태퍼,
구조 상태가 같은지 확인해야 합니다.")
        for name, value in file_peak_medians:
            print(f"    {name}: {value:.2f} Hz")

def score_features(model: dict, feat: pd.DataFrame, env: np.ndarray) ->
np.ndarray:
    x = model_features(model, feat)
    env_filled, _ = fill_env(env, np.asarray(model["env_medians"]))
    if model["env_model"] is not None:
        x = x - model["env_model"].predict(env_filled)
    scaled = model["scaler"].transform(x)
    recon = model["autoencoder"].predict(scaled)
    return np.mean((recon - scaled) ** 2, axis=1)

def wilson_interval(successes: int, total: int, z: float = 1.96) -> tuple[float,
float]:
    if total <= 0:
        return math.nan, math.nan
    p = successes / total
    denominator = 1.0 + z * z / total
    center = (p + z * z / (2.0 * total)) / denominator
    half = z * math.sqrt(p * (1.0 - p) / total + z * z / (4.0 * total * total)) /
denominator
    return max(0.0, center - half), min(1.0, center + half)

def evaluation_row(model: dict, bridge: Path, label: str) -> tuple[dict,
np.ndarray]:
    feat, env, _ = extract_events(bridge, model["delay"], model["window"])
    scores = score_features(model, feat, env)
    threshold = float(model["threshold"])
    ratios = scores / threshold
    detected = int(np.sum(scores > threshold))
    low, high = wilson_interval(detected, len(scores))
    row = {
        "condition": bridge.name.replace("_bridge_log.csv", ""),
        "label": label,
        "n": len(scores),
        "anomaly_count": detected,
        "detection_rate": detected / len(scores),
        "ci95_low": low,
        "ci95_high": high,
        "score_ratio_median": float(np.median(ratios)),
        "score_ratio_q1": float(np.quantile(ratios, 0.25)),
        "score_ratio_q3": float(np.quantile(ratios, 0.75)),
        "peak_hz_mean": float(feat["peak_hz"].mean()),
        "peak_hz_std": float(feat["peak_hz"].std(ddof=0)),
        "rms_mean": float(feat["rms"].mean()),
        "rms_std": float(feat["rms"].std(ddof=0)),
    }
    return row, scores

def evaluate(args: argparse.Namespace) -> None:
    model = joblib.load(args.model)
    rows = []
    normal_scores = []
    anomaly_scores = []
    stored_normal = np.asarray(model.get("normal_test_scores", []), dtype=float)
    if len(stored_normal):
        threshold = float(model["threshold"])

```

```

        detected = int(np.sum(stored_normal > threshold))
        low, high = wilson_interval(detected, len(stored_normal))
        ratios = stored_normal / threshold
        rows.append({
            "condition": "held_out_normal", "label": "normal", "n":
len(stored_normal),
            "anomaly_count": detected, "detection_rate": detected /
len(stored_normal),
            "ci95_low": low, "ci95_high": high,
            "score_ratio_median": float(np.median(ratios)),
            "score_ratio_q1": float(np.quantile(ratios, 0.25)),
            "score_ratio_q3": float(np.quantile(ratios, 0.75)),
            "peak_hz_mean": math.nan, "peak_hz_std": math.nan,
            "rms_mean": math.nan, "rms_std": math.nan,
        })
        normal_scores.extend(stored_normal.tolist())
    for value in args.normal:
        row, scores = evaluation_row(model, Path(value), "normal")
        rows.append(row)
        normal_scores.extend(scores.tolist())
    for value in args.anomaly:
        row, scores = evaluation_row(model, Path(value), "anomaly")
        rows.append(row)
        anomaly_scores.extend(scores.tolist())
    result = pd.DataFrame(rows)
    result.to_csv(args.output, index=False)
    if len(result):
        print(result.to_string(index=False, float_format=lambda value:
f"{value:.4f}"))
    print(f"결과 저장: {args.output}")
    if normal_scores and anomaly_scores:
        threshold = float(model["threshold"])
        normal_pred = np.asarray(normal_scores) > threshold
        anomaly_pred = np.asarray(anomaly_scores) > threshold
        tn, fp = int(np.sum(~normal_pred)), int(np.sum(normal_pred))
        tp, fn = int(np.sum(anomaly_pred)), int(np.sum(~anomaly_pred))
        precision = tp / (tp + fp) if tp + fp else math.nan
        recall = tp / (tp + fn) if tp + fn else math.nan
        f1 = 2 * precision * recall / (precision + recall) if precision + recall
else math.nan
        far = fp / (fp + tn) if fp + tn else math.nan
        print(f"TN={tn}, FP={fp}, FN={fn}, TP={tp}")
        print(f"FAR={far:.4f}, Precision={precision:.4f}, Recall={recall:.4f},
F1={f1:.4f}")

def tail_dataframe(path: Path, rows: int) -> pd.DataFrame:
    with path.open("rb") as f:
        header = f.readline().decode("utf-8").strip()
        f.seek(0, 2)
        pos = f.tell()
        blocks = []
        count = 0
        while pos > 0 and count <= rows:
            size = min(65536, pos)
            pos -= size
            f.seek(pos)
            block = f.read(size)
            blocks.append(block)
            count += block.count(b"\n")
        truncated = pos > 0
        raw = b"".join(reversed(blocks)).decode("utf-8", errors="ignore")
        lines = raw.splitlines()
        if lines and lines[0] == header:

```

```

        lines = lines[1:]
    elif truncated and lines:
        lines = lines[1:]
    lines = lines[-rows:]
    text = header + "\n" + "\n".join(lines)
    return pd.read_csv(io.StringIO(text))

def score_event(model: dict, seg: pd.DataFrame, env: np.ndarray) -> dict:
    config = model["config"]
    vector, peak, rms, freq, power = extract_one(seg, config["fs"], config["top"],
config["bands"])
    x = model_features(model, vector)
    env_filled, _ = fill_env(env.reshape(1, -1), np.asarray(model["env_medians"]))
    if model["env_model"] is not None:
        x = x - model["env_model"].predict(env_filled)
    scaled = model["scaler"].transform(x)
    recon = model["autoencoder"].predict(scaled)
    per_feature = (recon - scaled) ** 2
    score = float(np.mean(per_feature))
    threshold = float(model["threshold"])
    band_err = per_feature[0, 5:8]
    total = float(np.sum(band_err))
    bands = (band_err / total).tolist() if total > 0 else [0.0, 0.0, 0.0]
    return {"score": score, "score_ratio": score / threshold if threshold > 0 else
math.nan, "peak_hz": peak, "rms": rms, "freq_x": freq.tolist(), "freq_y":
power.tolist(), "bands": bands}

class Monitor:
    def __init__(self, bridge: Path, model_path: Path):
        self.bridge = bridge
        self.tap_path, _ = companion_paths(bridge)
        self.model = joblib.load(model_path)
        self.lock = threading.Lock()
        self.state = {"status_text": "대기 중", "status_class": "waiting",
"score_ratio": None, "peak_hz": None, "rms": None, "temp_c": None, "humidity":
None, "tap_index": None, "evaluated_count": 0, "anomaly_count": 0, "wave_x": [],
"wave_y": [], "freq_x": [], "freq_y": [], "bands": [0, 0, 0], "updated_at": None,
"info": "로거 데이터를 기다리는 중"}
        self.last_tap = -1
        self.evaluated_count = 0
        self.anomaly_count = 0
        threading.Thread(target=self.loop, daemon=True).start()

    def loop(self) -> None:
        while True:
            try:
                if not self.bridge.exists() or not self.tap_path.exists():
                    time.sleep(0.5)
                    continue
                config = self.model["config"]
                rows = int(config["fs"] * 15)
                data = tail_dataframe(self.bridge, rows)
                taps = read_taps(self.tap_path)
                if data.empty:
                    time.sleep(0.5)
                    continue
                t = pd.to_numeric(data["t_perf_ns"],
errors="coerce").to_numpy(float)
                z = pd.to_numeric(data["az_g"], errors="coerce").to_numpy(float)
                max_t = np.nanmax(t)
                recent_start = max_t - 2e9
                recent = t >= recent_start

```

```

        wave_y = z[recent]
        wave_x = ((t[recent] - t[recent][0]) / 1e9).tolist() if
np.any(recent) else []
        temp = float(pd.to_numeric(data["temp_c"],
errors="coerce").dropna().iloc[-1]) if "temp_c" in data and
pd.to_numeric(data["temp_c"], errors="coerce").notna().any() else None
        hum = float(pd.to_numeric(data["humidity"],
errors="coerce").dropna().iloc[-1]) if "humidity" in data and
pd.to_numeric(data["humidity"], errors="coerce").notna().any() else None
        unix_values = pd.to_numeric(data["t_unix_s"],
errors="coerce").dropna() if "t_unix_s" in data else pd.Series(dtype=float)
        updated_at = float(unix_values.iloc[-1]) if len(unix_values) else
time.time()
        update = {"wave_x": wave_x, "wave_y": wave_y.tolist(), "temp_c":
temp, "humidity": hum, "updated_at": updated_at}
        end_needed = taps["tap_ns"] + (self.model["delay"] +
self.model["window"]) * 1e9
        complete = taps[end_needed <= max_t]
        if len(complete):
            tap_idx = int(complete.index[-1])
            if tap_idx != self.last_tap:
                tap_ns = float(complete.iloc[-1]["tap_ns"])
                start = tap_ns + self.model["delay"] * 1e9
                end = start + self.model["window"] * 1e9
                m = (t >= start) & (t <= end)
                seg = data.loc[m]
                if len(seg) >= int(config["fs"] * self.model["window"] *
0.7):
                    env = np.array([float(pd.to_numeric(seg["temp_c"],
errors="coerce").mean()) if "temp_c" in seg else math.nan,
float(pd.to_numeric(seg["humidity"], errors="coerce").mean()) if "humidity" in seg
else math.nan])
                    result = score_event(self.model, seg, env)
                    ratio = result["score_ratio"]
                    if not np.isfinite(ratio):
                        text, css = "측정 대기", "waiting"
                    elif ratio < 0.8:
                        text, css = "정상 범위", "normal"
                    elif ratio < 1.0:
                        text, css = "주의", "warning"
                    else:
                        text, css = "이상 후보", "anomaly"
                    self.evaluated_count += 1
                    if np.isfinite(ratio) and ratio >= 1.0:
                        self.anomaly_count += 1
                    env_text = "환경 보정 사용" if
self.model.get("environment_enabled", self.model.get("env_model") is not None)
else "환경 보정 미사용"
                    update.update(result)
                    update.update({"status_text": text, "status_class":
css, "tap_index": tap_idx, "evaluated_count": self.evaluated_count,
"anomaly_count": self.anomaly_count, "info": f"Autoencoder 8-8-3-8-8 | 로그·대역비율
피쳐 | {env_text} | 분석 대역 0-{config['top']:.1f} Hz | 임계값
{self.model['threshold']:.5f} | 최근 타격 {tap_idx}"})
                    self.last_tap = tap_idx
                with self.lock:
                    self.state.update(update)
            except Exception as exc:
                with self.lock:
                    self.state["info"] = f"대기/오류: {exc}"
        time.sleep(0.5)

def get(self) -> dict:
    with self.lock:

```

```

        return dict(self.state)

def local_ip() -> str:
    try:
        result = subprocess.run(["hostname", "-I"], capture_output=True, text=True,
check=False)
        for value in result.stdout.split():
            if ":" not in value and not value.startswith("127."):
                return value
    except OSError:
        pass
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        sock.connect(("10.255.255.255", 1))
        value = sock.getsockname()[0]
        sock.close()
        return value
    except OSError:
        return "라즈베리파이_IP"

def serve(bridge: Path, model: Path, host: str, port: int) -> None:
    monitor = Monitor(bridge, model)
    app = Flask(__name__)

    @app.get("/")
    def index():
        return HTML

    @app.get("/api")
    def api():
        return jsonify(monitor.get())

    print(f"대시보드: http://{local_ip()}:{port}")
    app.run(host=host, port=port, threaded=True, use_reloader=False)

def run(args: argparse.Namespace) -> None:
    output = Path(args.output_dir)
    output.mkdir(parents=True, exist_ok=True)
    bridge = output / f"{args.condition}_bridge_log.csv"
    tap_path, meta_path = companion_paths(bridge)
    for path in (bridge, tap_path, meta_path):
        if path.exists():
            path.unlink()
    logger = Path(args.logger)
    if not logger.is_absolute() and not logger.exists():
        candidate = Path(__file__).resolve().parent / logger
        if candidate.exists():
            logger = candidate
    model = Path(args.model)
    if not model.exists():
        raise FileNotFoundError(f"모델 파일을 찾지 못했습니다: {model}")
    command = [sys.executable, str(logger), "--condition", args.condition,
"--output-dir", str(output), "--sample-rate", str(args.sample_rate), "--range-g",
str(args.range_g), "--pulse-ms", str(args.pulse_ms), "--period-s",
str(args.period_s), "--warmup", str(args.warmup), "--n-taps", str(args.n_taps)]
    process = subprocess.Popen(command)
    try:
        deadline = time.time() + args.warmup * args.period_s + 30
        while not bridge.exists() and time.time() < deadline:
            if process.poll() is not None:
                raise RuntimeError("로거가 먼저 종료되었습니다.")

```

```

        time.sleep(0.5)
    if not bridge.exists():
        raise RuntimeError("bridge CSV가 생성되지 않았습니다.")
    serve(bridge, model, args.host, args.port)
finally:
    if process.poll() is None:
        process.terminate()

def parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser()
    sub = p.add_subparsers(dest="mode", required=True)
    t = sub.add_parser("train")
    t.add_argument("--bridge", nargs="+", required=True)
    t.add_argument("--model", default="sentry_autoencoder.joblib")
    t.add_argument("--delay", type=float, default=0.05)
    t.add_argument("--window", type=float, default=2.0)
    t.add_argument("--train-count", type=int, default=120)
    t.add_argument("--threshold-count", type=int, default=30)
    t.add_argument("--test-count", type=int, default=30)
    t.add_argument("--seed", type=int, default=42)
    e = sub.add_parser("evaluate")
    e.add_argument("--model", default="sentry_autoencoder.joblib")
    e.add_argument("--normal", nargs="*", default=[])
    e.add_argument("--anomaly", nargs="*", default=[])
    e.add_argument("--output", default="evaluation_summary.csv")
    d = sub.add_parser("dashboard")
    d.add_argument("--bridge", required=True)
    d.add_argument("--model", default="sentry_autoencoder.joblib")
    d.add_argument("--host", default="0.0.0.0")
    d.add_argument("--port", type=int, default=8080)
    r = sub.add_parser("run")
    r.add_argument("--logger", default="logger3_fixed.py")
    r.add_argument("--model", default="sentry_autoencoder.joblib")
    r.add_argument("--condition", default="T0-H0-M0-B0")
    r.add_argument("--output-dir", default="data/live")
    r.add_argument("--sample-rate", type=int, default=500)
    r.add_argument("--range-g", type=int, default=8)
    r.add_argument("--pulse-ms", type=float, default=40)
    r.add_argument("--period-s", type=float, default=5)
    r.add_argument("--warmup", type=int, default=10)
    r.add_argument("--n-taps", type=int, default=35)
    r.add_argument("--host", default="0.0.0.0")
    r.add_argument("--port", type=int, default=8080)
    return p

def main() -> None:
    args = parser().parse_args()
    if args.mode == "train":
        train(args)
    elif args.mode == "evaluate":
        evaluate(args)
    elif args.mode == "dashboard":
        serve(Path(args.bridge), Path(args.model), args.host, args.port)
    else:
        run(args)

if __name__ == "__main__":
    main()

```

logger3_fixed.py

```
from __future__ import annotations

import argparse
import csv
import json
import math
import multiprocessing as mp
import queue
import threading
import time
from pathlib import Path
from typing import Final

import adafruit_dht
import board
from gpiozero import OutputDevice
from smbus2 import SMBus

ADXL355_ADDR_LOW: Final = 0x1D
ADXL355_ADDR_HIGH: Final = 0x53

REG_DEVID_AD: Final = 0x00
REG_TEMP2: Final = 0x06
REG_XDATA3: Final = 0x08
REG_FILTER: Final = 0x28
REG_RANGE: Final = 0x2C
REG_POWER_CTL: Final = 0x2D
REG_RESET: Final = 0x2F

EXPECTED_ID: Final = (0xAD, 0x1D, 0xED)

RANGE_TO_CODE = {2: 0x01, 4: 0x02, 8: 0x03}
RANGE_TO_SCALE_G_PER_LSB = {
    2: 2.048 / (2**19),
    4: 4.096 / (2**19),
    8: 8.192 / (2**19),
}
RANGE_TO_FULL_SCALE_G = {2: 2.048, 4: 4.096, 8: 8.192}

SAMPLE_RATE_TO_FILTER = {125: 0x05, 250: 0x04, 500: 0x03}

TEMP_BIAS_LSB: Final = 1852.0
TEMP_SLOPE_LSB_PER_C: Final = -9.05

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="SENTRY ADXL355 + DHT22 + solenoid tapper logger"
    )
    parser.add_argument(
        "--condition",
        default="Tx-Hx-M0-B0",
        help="조건 코드. 예: T0-H0-M500-B0",
    )
    parser.add_argument("--output-dir", default="data", help="CSV 저장 폴더")
    parser.add_argument("--bus", type=int, default=1, help="I2C 버스 번호")
    parser.add_argument(
        "--address",
        choices=["auto", "0x1d", "0x53"],
    )
```

```

        default="auto",
        help="ADXL355 I2C 주소",
    )
    parser.add_argument(
        "--sample-rate",
        type=int,
        choices=sorted(SAMPLE_RATE_TO_FILTER),
        default=500,
        help="ADXL355 ODR와 로거 목표 샘플링 주파수. 최종 실험 전체에서 고정",
    )
    parser.add_argument(
        "--range-g",
        type=int,
        choices=[2, 4, 8],
        default=8,
        help="가속도 측정 범위",
    )
    parser.add_argument(
        "--dht-interval",
        type=float,
        default=2.0,
        help="DHT22 측정 간격(초)",
    )
    parser.add_argument(
        "--adxl-temp-interval",
        type=float,
        default=1.0,
        help="ADXL355 내부 온도 측정 간격(초)",
    )
    parser.add_argument(
        "--pulse-ms",
        type=float,
        default=40.0,
        help="태퍼 펄스폭(ms). 예비 실험에서 포화가 없도록 확정",
    )
    parser.add_argument(
        "--period-s",
        type=float,
        default=5.0,
        help="타격 시작 시점 간 주기(초)",
    )
    parser.add_argument(
        "--warmup",
        type=int,
        default=10,
        help="기록 전에 같은 펄스·같은 주기로 수행할 워밍업 타격 횟수",
    )
    parser.add_argument(
        "--pre-tap-seconds",
        type=float,
        default=2.0,
        help="기록 시작 후 첫 본 타격까지의 대기 시간",
    )
    parser.add_argument("--n-taps", type=int, default=35, help="본 타격 횟수")
    parser.add_argument(
        "--tail-seconds",
        type=float,
        default=5.0,
        help="마지막 타격 후 추가 수집 시간",
    )
    parser.add_argument(
        "--flush-seconds",
        type=float,
        default=1.0,

```

```

        help="CSV 디스크 flush 간격",
    )
    parser.add_argument(
        "--tapper-gpio",
        type=int,
        default=17,
        help="솔레노이드 드라이버 제어 BCM GPIO",
    )
    parser.add_argument(
        "--no-tapper",
        action="store_true",
        help="테퍼 없이 연속 수집. Ctrl+C로 종료",
    )
    parser.add_argument(
        "--no-dht",
        action="store_true",
        help="DHT22 없이 가속도만 수집",
    )
    return parser.parse_args()

def twos_complement(value: int, bits: int) -> int:
    sign_bit = 1 << (bits - 1)
    return value - (1 << bits) if value & sign_bit else value

def detect_address(bus: SMBus, requested: str) -> int:
    candidates = (
        [ADXL355_ADDR_LOW, ADXL355_ADDR_HIGH]
        if requested == "auto"
        else [int(requested, 16)]
    )
    for address in candidates:
        try:
            device_id = tuple(bus.read_i2c_block_data(address, REG_DEVID_AD, 3))
        except OSError:
            continue
        if device_id == EXPECTED_ID:
            return address
    raise RuntimeError(
        "ADXL355를 찾지 못했습니다. i2cdetect -y 1에서 1d 또는 53이 "
        "보이는지 확인하세요."
    )

def configure_adxl355(
    bus: SMBus, address: int, range_g: int, sample_rate: int
) -> None:
    bus.write_byte_data(address, REG_RESET, 0x52)
    time.sleep(0.1)

    bus.write_byte_data(address, REG_RANGE, 0x80 | RANGE_TO_CODE[range_g])
    bus.write_byte_data(address, REG_FILTER, SAMPLE_RATE_TO_FILTER[sample_rate])
    bus.write_byte_data(address, REG_POWER_CTL, 0x00)
    time.sleep(0.1)

def read_adxl_temperature_c(bus: SMBus, address: int) -> float:
    temp2, temp1 = bus.read_i2c_block_data(address, REG_TEMP2, 2)
    raw = ((temp2 & 0x0F) << 8) | temp1
    return 25.0 + (raw - TEMP_BIAS_LSB) / TEMP_SLOPE_LSB_PER_C

```

```

def read_acceleration_g(
    bus: SMBus, address: int, scale_g_per_lsb: float
) -> tuple[float, float, float]:
    data = bus.read_i2c_block_data(address, REG_XDATA3, 9)
    values: list[float] = []
    for i in range(0, 9, 3):
        raw20 = (data[i] << 12) | (data[i + 1] << 4) | (data[i + 2] >> 4)
        values.append(twos_complement(raw20, 20) * scale_g_per_lsb)
    return values[0], values[1], values[2]

def wait_until_ns(target_ns: int) -> None:
    while True:
        remaining_ns = target_ns - time.perf_counter_ns()
        if remaining_ns <= 0:
            return
        if remaining_ns > 2_000_000:
            time.sleep((remaining_ns - 1_000_000) / 1e9)
        else:
            time.sleep(0)

def pulse_tapper(tap: OutputDevice, pulse_seconds: float) -> tuple[int, int]:
    start_ns = time.perf_counter_ns()
    tap.on()
    time.sleep(pulse_seconds)
    tap.off()
    end_ns = time.perf_counter_ns()
    return start_ns, end_ns

def run_warmup(
    tap: OutputDevice,
    warmup_count: int,
    period_s: float,
    pulse_s: float,
) -> None:
    if warmup_count <= 0:
        return
    print(
        f"[태퍼] 워밍업 {warmup_count}회: 본 실험과 같은 "
        f"{period_s:.3f} s 주기, {pulse_s * 1000:.1f} ms 펄스"
    )
    start_ns = time.perf_counter_ns() + 1_000_000_000
    period_ns = int(period_s * 1e9)
    for i in range(warmup_count):
        wait_until_ns(start_ns + i * period_ns)
        pulse_tapper(tap, pulse_s)

def unix_from_perf_ns(
    perf_ns: int, anchor_perf_ns: int, anchor_unix_s: float
) -> float:
    return anchor_unix_s + (perf_ns - anchor_perf_ns) / 1e9

def dht_worker(
    output_queue: mp.Queue,
    stop_event: mp.Event,
    interval_s: float,
) -> None:
    dht = adafruit_dht.DHT22(board.D4)
    try:

```

```

while not stop_event.is_set():
    measured_ns = time.perf_counter_ns()
    temp_c = math.nan
    humidity = math.nan
    try:
        temp = dht.temperature
        hum = dht.humidity
        if temp is not None:
            temp_c = float(temp)
        if hum is not None:
            humidity = float(hum)
    except RuntimeError:
        pass

    item = (measured_ns, temp_c, humidity)
    try:
        while True:
            output_queue.get_nowait()
    except queue.Empty:
        pass
    try:
        output_queue.put_nowait(item)
    except queue.Full:
        pass

    stop_event.wait(interval_s)
finally:
    dht.exit()

```

```

def tapper_worker(
    args: argparse.Namespace,
    tap: OutputDevice,
    tap_path: Path,
    stop_event: threading.Event,
    anchor_perf_ns: int,
    anchor_unix_s: float,
    first_tap_ns: int,
) -> None:
    pulse_s = args.pulse_ms / 1000.0
    period_ns = int(args.period_s * 1e9)

    try:
        with tap_path.open("w", newline="", encoding="utf-8") as file:
            writer = csv.writer(file)
            writer.writerow(
                [
                    "tap_index",
                    "scheduled_perf_ns",
                    "pulse_start_perf_ns",
                    "pulse_end_perf_ns",
                    "pulse_start_unix_s",
                    "start_delay_ms",
                    "condition",
                ]
            )

        print(
            f"[태퍼] 본 타격 {args.n_taps}회 시작 "
            f"({args.period_s:.3f} s start-to-start)"
        )
        for index in range(args.n_taps):
            scheduled_ns = first_tap_ns + index * period_ns
            wait_until_ns(scheduled_ns)
    
```

```

        start_ns, end_ns = pulse_tapper(tap, pulse_s)
        writer.writerow(
            [
                index,
                scheduled_ns,
                start_ns,
                end_ns,
                f"{unix_from_perf_ns(start_ns, anchor_perf_ns,
anchor_unix_s):.9f}",
                f"((start_ns - scheduled_ns) / 1e6:.3f)",
                args.condition,
            ]
        )
        file.flush()

    print(
        f"[태퍼] 완료. 마지막 타격 후 {args.tail_seconds:.2f} s 추가 수집"
    )
    time.sleep(args.tail_seconds)
finally:
    stop_event.set()

def format_optional(value: float, digits: int = 3) -> str:
    return "" if math.isnan(value) else f"{value:.{digits}f}"

def main() -> None:
    args = parse_args()
    if args.period_s <= args.pulse_ms / 1000.0:
        raise ValueError("period-s는 pulse-ms보다 길어야 합니다.")

    output_dir = Path(args.output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    accel_path = output_dir / f"{args.condition}_bridge_log.csv"
    tap_path = output_dir / f"{args.condition}_tap_events.csv"
    metadata_path = output_dir / f"{args.condition}_metadata.json"

    scale = RANGE_TO_SCALE_G_PER_LSB[args.range_g]
    full_scale_g = RANGE_TO_FULL_SCALE_G[args.range_g]
    clip_limit_g = 0.995 * full_scale_g
    sample_interval_ns = int(1e9 / args.sample_rate)

    tap: OutputDevice | None = None
    tapper_thread: threading.Thread | None = None
    dht_process: mp.Process | None = None
    dht_queue: mp.Queue | None = None
    dht_stop: mp.Event | None = None
    stop_event = threading.Event()

    latest_dht_temp = math.nan
    latest_humidity = math.nan
    latest_dht_perf_ns = 0
    latest_adxl_temp = math.nan
    last_adxl_temp_ns = 0

    total_samples = 0
    missed_slots = 0
    clip_counts = {"ax": 0, "ay": 0, "az": 0}

    try:
        if not args.no_tapper:
            tap = OutputDevice(

```

```

        args.tapper_gpio, active_high=True, initial_value=False
    )

    if not args.no_dht:
        ctx = mp.get_context("spawn")
        dht_queue = ctx.Queue(maxsize=1)
        dht_stop = ctx.Event()
        dht_process = ctx.Process(
            target=dht_worker,
            args=(dht_queue, dht_stop, args.dht_interval),
            daemon=True,
        )
        dht_process.start()

    with SMBus(args.bus) as bus:
        address = detect_address(bus, args.address)
        configure_adxl355(
            bus, address, args.range_g, args.sample_rate
        )

    if not args.no_tapper:
        if tap is None:
            raise RuntimeError("테퍼 GPIO 초기화에 실패했습니다.")
        run_warmup(
            tap,
            args.warmup,
            args.period_s,
            args.pulse_ms / 1000.0,
        )

    anchor_perf_ns = time.perf_counter_ns()
    anchor_unix_s = time.time()
    first_tap_ns = (
        anchor_perf_ns + int(args.pre_tap_seconds * 1e9)
    )

    metadata = {
        "condition": args.condition,
        "address_hex": f"0x{address:02X}",
        "sample_rate_hz": args.sample_rate,
        "filter_register_hex": (
            f"0x{SAMPLE_RATE_TO_FILTER[args.sample_rate]:02X}"
        ),
        "range_g": args.range_g,
        "pulse_ms": args.pulse_ms,
        "period_s": args.period_s,
        "warmup_taps": args.warmup,
        "recorded_taps": 0 if args.no_tapper else args.n_taps,
        "pre_tap_seconds": args.pre_tap_seconds,
        "tail_seconds": args.tail_seconds,
        "start_unix_s": anchor_unix_s,
    }

    metadata_path.write_text(
        json.dumps(metadata, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

    print(f"[수집] 조건: {args.condition}")
    print(
        f"[수집] ADXL355 0x{address:02X}, "
        f"±{args.range_g} g, {args.sample_rate} Hz"
    )

    print(f"[수집] 가속도 CSV: {accel_path}")
    if not args.no_tapper:

```

```

        print(f"[수집] 타격 CSV: {tap_path}")

    if not args.no_tapper:
        tapper_thread = threading.Thread(
            target=tapper_worker,
            args=(
                args,
                tap,
                tap_path,
                stop_event,
                anchor_perf_ns,
                anchor_unix_s,
                first_tap_ns,
            ),
            daemon=True,
        )
        tapper_thread.start()

    next_sample_ns = time.perf_counter_ns()
    next_flush_ns = (
        next_sample_ns + int(args.flush_seconds * 1e9)
    )

    with accel_path.open(
        "w", newline="", encoding="utf-8", buffering=1024 * 1024
    ) as file:
        writer = csv.writer(file)
        writer.writerow(
            [
                "sample_index",
                "t_perf_ns",
                "t_unix_s",
                "scheduled_perf_ns",
                "lateness_us",
                "missed_slots_total",
                "temp_c",
                "humidity",
                "dht_age_s",
                "adxl_temp_c",
                "ax_g",
                "ay_g",
                "az_g",
            ]
        )

    while not stop_event.is_set():
        wait_until_ns(next_sample_ns)
        now_ns = time.perf_counter_ns()

        if now_ns - next_sample_ns >= sample_interval_ns:
            skipped = int(
                (now_ns - next_sample_ns) // sample_interval_ns
            )
            missed_slots += skipped
            next_sample_ns += skipped * sample_interval_ns

        scheduled_ns = next_sample_ns

        if dht_queue is not None:
            try:
                while True:
                    (
                        latest_dht_perf_ns,

```

```

        temp_value,
        humidity_value,
    ) = dht_queue.get_nowait()
    if not math.isnan(temp_value):
        latest_dht_temp = temp_value
    if not math.isnan(humidity_value):
        latest_humidity = humidity_value
except queue.Empty:
    pass

if (
    now_ns - last_adxl_temp_ns
    >= int(args.adxl_temp_interval * 1e9)
):
    try:
        latest_adxl_temp = read_adxl_temperature_c(
            bus, address
        )
    except OSError:
        latest_adxl_temp = math.nan
    last_adxl_temp_ns = now_ns

read_start_ns = time.perf_counter_ns()
ax_g, ay_g, az_g = read_acceleration_g(
    bus, address, scale
)
read_end_ns = time.perf_counter_ns()
sample_perf_ns = (read_start_ns + read_end_ns) // 2

for key, value in (
    ("ax", ax_g),
    ("ay", ay_g),
    ("az", az_g),
):
    if abs(value) >= clip_limit_g:
        clip_counts[key] += 1

dht_age_s = (
    math.nan
    if latest_dht_perf_ns == 0
    else (sample_perf_ns - latest_dht_perf_ns) / 1e9
)

writer.writerow(
    [
        total_samples,
        sample_perf_ns,
        f"{unix_from_perf_ns(sample_perf_ns, anchor_perf_ns,
anchor_unix_s):.9f}",
        scheduled_ns,
        f"{(sample_perf_ns - scheduled_ns) / 1000.0:.3f}",
        missed_slots,
        format_optional(latest_dht_temp),
        format_optional(latest_humidity),
        format_optional(dht_age_s, 3),
        format_optional(latest_adxl_temp),
        f"{ax_g:.8f}",
        f"{ay_g:.8f}",
        f"{az_g:.8f}",
    ]
)
total_samples += 1
next_sample_ns += sample_interval_ns

```

```

        if read_end_ns >= next_flush_ns:
            file.flush()
            next_flush_ns = (
                read_end_ns + int(args.flush_seconds * 1e9)
            )

except KeyboardInterrupt:
    print("\n[수집] Ctrl+C로 중단됨")
finally:
    stop_event.set()

    if tapper_thread is not None:
        tapper_thread.join(timeout=3.0)

    if dht_stop is not None:
        dht_stop.set()
    if dht_process is not None:
        dht_process.join(timeout=3.0)
        if dht_process.is_alive():
            dht_process.terminate()

    if tap is not None:
        tap.off()
        tap.close()

    print("[수집] 종료")
    print(f"[QC] 저장 샘플: {total_samples}")
    print(f"[QC] 건너편 샘플 슬롯: {missed_slots}")
    print(
        "[QC] 포화 샘플 수: "
        f"ax={clip_counts['ax']}, "
        f"ay={clip_counts['ay']}, "
        f"az={clip_counts['az']}"
    )
    if any(clip_counts.values()):
        print(
            "[경고] 포화가 발생했습니다. pulse-ms를 줄이거나 "
            "range-g를 높인 뒤 재수집하세요."
        )

if __name__ == "__main__":
    main()

```